A3: Activation-Aware Factorized SVD for Transformer Compression

Taha Majs* **Research Team**
Department of Computer Science
Advanced Machine Learning & Efficiency Laboratory

September 18, 2026

Abstract

Low-rank factorization is a classical approach to compressing the linear layers of transformer models, replacing each weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with a product UV where $U \in \mathbb{R}^{d_{\text{out}} \times r}$, $V \in \mathbb{R}^{r \times d_{\text{in}}}$, and $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Standard SVD applies this factorization uniformly across all input dimensions, ignoring the fact that transformer activations have highly non-uniform variance across channels. We introduce **A3** (**A**ctivation-**A**ware **A**daptive factorization), which addresses this by (i) collecting a streaming empirical Fisher diagonal during calibration, (ii) weighting the SVD input space by this energy estimate before factorization, (iii) adaptively increasing rank when reconstruction error exceeds a threshold, and (iv) recalibrating LayerNorm affine parameters after factorization to absorb distributional shift.

A3 achieves guaranteed parameter reduction without expansion (+1.25M parameters saved on DistilBERT, +0.59M on RoBERTa, +0.53M on GPT-2) under an enforced rank ceiling $r_{\text{ceiling}} \leq 256$. On SST-2, A3 preserves 90.02% validation accuracy on DistilBERT ($\Delta -$ 0.69%, F1: 0.9043) and achieves 83.26% on RoBERTa ($\Delta + 4.70\%$, F1: 0.8460), passing quality gates on DistilBERT and RoBERTa, while GPT-2 shows significant degradation (-16.17%, 63.19% accuracy, F1: 0.7216) that requires further investigation. We additionally evaluate on MNLI and AG News, finding that A3 requires task-specific base checkpoints to achieve meaningful accuracy on multi-class tasks. We provide an open-source, fully reproducible implementation with strict Frobenius-norm validation, zero-overhead CPU FP32 SVD factorization, and automated Hugging Face hub integration.

Keywords: low-rank factorization, activation-aware compression, Fisher information, LayerNorm recalibration, transformer efficiency

*Corresponding author: tahamajs@uni.edu

1 Introduction

1.1 Motivation

The quadratic parameter cost of transformer layers makes deployment of even moderately-sized models expensive. A standard BERT-base layer contains approximately 7 million parameters, of which 4.7 million (67%) live in the four attention projections W_Q, W_K, W_V, W_O and the remaining 2.4 million in the FFN. For DistilBERT, this drops to 3.5M and 2.4M respectively. However, even DistilBERT (66.9M parameters) requires 134 MB of storage at FP16, which is not negligible for mobile or edge deployment.

Low-rank factorization is the classical approach to reducing this cost. Given a linear layer $y = xW^\top + b$, one replaces W with a product $W \approx UV$ where $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{r \times d_{\text{in}}}$, $r \ll \min(d_{\text{in}}, d_{\text{out}})$. The parameter count decreases from $d_{\text{in}} d_{\text{out}}$ to $r(d_{\text{in}} + d_{\text{out}})$, achieving compression ratio $\frac{r(d_{\text{in}}+d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}$. For a 768×768 matrix at rank $r = 64$, this is a $5.9\times$ reduction (589,824 $\rightarrow$ 98,304 parameters).

Classical SVD is optimal in an unweighted Frobenius norm sense:

$$W_r^* = \arg \min_{W_r : \text{rank}(W_r) = r} \|W - W_r\|_F \tag{1.1}$$

but this optimality is with respect to uniform treatment of all input dimensions. Transformer activations, however, have **highly non-uniform variance** across channels. Some input dimensions carry most of the signal; others are nearly constant. Reconstructing near-constant dimensions as faithfully as high-variance channels wastes capacity. Activation-aware compression addresses this fundamental limitation by weighting the reconstruction objective per input dimension according to its empirical importance. The natural weight is the empirical Fisher diagonal, $F_i = \mathbb{E}_{\mathbf{x}}[(\nabla_W \mathcal{L})_i^2] \approx \mathbb{E}_{\mathbf{x}}[x_i^2]$, which measures the second-order sensitivity and variance attributable to input channel i. Reweighting the SVD input space by this diagonal quantity concentrates the low-rank approx-

imation capacity precisely onto the dominant feature directions that govern downstream task representations.

Given a pretrained transformer model $\mathcal{M}$ with N_l layers, each containing linear projections $\{W_k^{(\ell)}\}$, we seek a compressed model $\mathcal{M}'$ such that:

$$\mathcal{M}' \approx \mathcal{M}, \qquad \frac{|\theta(\mathcal{M}')|}{|\theta(\mathcal{M})|} \ll 1 \tag{1.2}$$

where $|\theta(\cdot)|$ denotes parameter count.

We impose four core constraints on the compression pipeline to ensure production reliability. First, we enforce a strict reconstruction error bound: for each factorized layer, the normalized Frobenius reconstruction error must satisfy $\|W - UV\|_F / \|W\|_F \leq \tau$ with safety threshold $\tau = 0.40$. Second, we forbid retraining of original base weights from scratch, allowing only parameter-efficient LoRA adapter recovery to preserve the compute budget. Third, all activation statistics must be gathered under a memory-bounded streaming protocol operating in $\mathcal{O}(d^2)$ auxiliary memory rather than $\mathcal{O}(Nd^2)$ batch storage. Finally, the system mandates deterministic fallback, ensuring that any projection violating the error threshold seamlessly retains its original uncompressed dense weights without corrupting the representation pipeline.

1.2 Our Approach

To satisfy these objectives, we propose **A3** (**A**ctivation-**A**ware **A**daptive factorization), a framework that departs from conventional truncated SVD across four foundational dimensions. First, A3 utilizes empirical Fisher-diagonal weighting by estimating the sensitivity vector $F_i = \mathbb{E}_x[x_i^2]$ from calibration tokens and reweighting the input space via $W_{\text{weighted}} = W \cdot \text{diag}(\sqrt{F + \epsilon})$ to protect high-impact feature coordinates. Second, online streaming covariance accumulation gathers both the diagonal Fisher vector F and the outer-product matrix C_X in constant $\mathcal{O}(d^2)$ working memory regardless of calibration sequence count. Third, adaptive rank escalation dynamically adjusts the allocated low-rank budget: when initial factorization exceeds the error threshold τ, the algorithm progressively increments the candidate rank up to $\min(d_{\text{in}}, d_{\text{out}})$, re-evaluating the factorization until convergence or deterministic fallback occurs. Fourth, post-factorization LayerNorm recalibration rescales the affine parameter γ according to the ratio of observed to target standard deviations ($\sigma_{\text{current}}/\sigma_{\text{target}}$), effectively compensating for activation variance shrinkage induced by truncation.

1.3 Contributions

In summary, this work makes four primary contributions. First, we propose A3, a streaming activation-aware low-rank compression framework that synergistically integrates empirical Fisher diagonal weighting, constant $\mathcal{O}(d^2)$ streaming covariance accumulation, adaptive rank escalation, and closed-form LayerNorm affine recalibration. Second, we deliver a production-hardened PyTorch implementation supporting flat, nested, and causal transformer architectures with deterministic fallback and zero-transient-memory CPU FP32 SVD factorization. Third, we establish comprehensive empirical benchmarks across DistilBERT, RoBERTa, and GPT-2 on SST-2 alongside multi-class stress tests on MNLI and AG News, rigorously dissecting the impact of rank ceilings, subspace weighting, and normalization dynamics. Finally, we provide full open-source reproducibility, releasing all factorization code, fine-tuning recipes, and model checkpoints under an open MIT license.

1.4 Paper Organization

Section 2 reviews related work in low-rank compression, activation-aware pruning, and LayerNorm recalibration. Section 3 provides background on SVD, the Fisher information, and streaming statistics. Section 4 presents A3. Section 5 describes integration with three transformer architectures. Section 6 details the experimental setup. Section 7 reports results. Section 8 analyzes where A3 succeeds and fails. Section 9 discusses practical guidance. Section 10 lists limitations. Section 11 concludes.

1.5 Notation

2 Related Work

2.1 Low-Rank Matrix Factorization

Classical SVD. The Eckart–Young theorem (Eckart & Young, 1936) guarantees that the best rank-r approximation of W in Frobenius norm is the truncated SVD:

$$W_r^* = U_{:,:r} \Sigma_{:,:r} V_{:r,:}^\top \tag{2.1}$$

This is optimal in an unweighted sense and forms the basis of most SVD-based compression methods.

Randomized SVD. Halko et al. (2011) provide a randomized algorithm for truncated SVD with complexity $\mathcal{O}(d_{\text{in}} d_{\text{out}} \log r)$, which we do not use because exact SVD is feasible for the layer sizes in our study.

Weighted low-rank approximation. Golub et al. (1987) and Srebro & Jaakkola (2003) study weighted variants of low-rank approximation. Our Fisher weighting is a diagonal special case with input-dimension-dependent weights.

2.2 Transformer Compression

2.2.1 Post-Training Pruning

Magnitude pruning (Han et al., 2015) removes individual weights. Structured pruning (Li et al., 2017) removes

entire channels. SparseGPT (Frantar & Alistarh, 2023) and Wanda (Sun et al., 2024) prune large language models without retraining. A3 differs by factorizing rather than zeroing.

2.2.2 Knowledge Distillation

DistilBERT (Sanh et al., 2019), TinyBERT (Jiao et al., 2020), and MiniLM (Wang et al., 2020) train smaller models to mimic larger ones. A3 is orthogonal – it can compress an already-distilled model.

2.2.3 Quantization

8-bit (Dettmers et al., 2022) and 4-bit (Dettmers et al., 2023) quantization reduces bit-width. GPTQ (Frantar et al., 2023) uses second-order information. A3 is orthogonal to quantization – the two can be combined.

2.2.4 Low-Rank Factorization

ALBERT (Lan et al., 2020) uses cross-layer parameter sharing, a form of low-rank structure. Tensor train decomposition (Novikov et al., 2015) uses higher-order factorization. **A3 differs by using activation-aware weighting during factorization**, not after.

2.3 Activation-Aware Compression

AWQ. Lin et al. (2023) find that activation magnitude correlates with weight importance. They use it for quantization. Our Fisher weighting uses the same insight for low-rank factorization.

Activation-aware pruning. Molchanov et al. (2019) use the Taylor expansion of the loss with respect to activations. Our approach is simpler: we use second moments.

Fisher information. The Fisher diagonal is the classical measure of parameter importance (Pascanu & Bengio, 2014; Kirkpatrick et al., 2017). We use a cheap empirical proxy $\mathbb{E}[x^2]$ instead of the full Fisher.

2.4 LayerNorm Recalibration

Post-training recalibration. Nagel et al. (2019) recalibrate activation distributions for quantization. We adapt this idea to post-factorization distributional shift.

Gamma rescaling. Gholami et al. (2021) rescale LayerNorm scales after quantization. Our $W^* = U\Sigma V^T$ is the analogous operation for low-rank factorization.

2.5 Position of Our Work

Axis	Prior Work	A3 (Ours)
Weighting	Uniform (unweighted)	Fisher diagonal
Rank Selection	Fixed global	Adaptive escalation
Calibration Mem.	Buffer $\mathcal{O}(Nd)$	Streaming $\mathcal{O}(d^2)$
Post-Factor Fix	None	LN recalibration
Error Validation	Silent	Strict Frobenius
Safety Fallback	Never	Automatic fallback

Table 1: Comparison of architectural axes between prior low-rank methods and A3.

Key distinction. Prior low-rank compression methods either (a) select rank globally before factorizing, or (b) factorize unweighted. A3 combines both improvements: it **selects rank per-layer based on validation**, and it **weights the factorization by activation statistics**. The result is strictly smaller reconstruction error at the same rank, or equivalently, smaller rank at the same error.

2.6 Comparison Table

3 Background

3.1 Singular Value Decomposition

For any matrix $W \in \mathbb{R}^{m\times n}$, the SVD is:

$$W = U\Sigma V^\top \tag{3.1}$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal, and $\Sigma \in \mathbb{R}^{m\times n}$ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

Truncated SVD. Keeping the top-r singular values gives the best rank-r approximation in Frobenius norm:

$$W_r = U_{:,:r}\Sigma_{:,:r}V^\top_{:r,:} \tag{3.2}$$

with squared error $\|W - W_r\|_F^2 = \sum_{i>r}\sigma_i^2$, or equivalently $\|W - W_r\|_F = \sqrt{\sum_{i>r}\sigma_i^2}$.

Symmetric factorization. For numerical stability, it is common to split $\Sigma^{1/2}$ between the two factors:

$$U_r = U_{:,:r}\Sigma^{1/2}_{:,:r}, \qquad V_r = \Sigma^{1/2}_{:r,:}V^\top_{:r,:} \tag{3.3}$$

so that $W_r = U_r V_r$.

3.2 The Fisher Information Matrix

For a probabilistic model $p_\theta(y|x)$, the Fisher information is:

$$\mathcal{F}_{ij} = \mathbb{E}_{x,y}\left[\frac{\partial \log p_\theta(y|x)}{\partial\theta_i}\cdot\frac{\partial \log p_\theta(y|x)}{\partial\theta_j}\right] \tag{3.4}$$

For a linear layer $y = Wx + b$, the Fisher with respect to x is:

Method	Type	Year	Venue	Model	Ratio	SST-2 Acc
SVD (Eckart-Young)[1]	Low-rank	1936	Psychometrika	DistilBERT	1.25×[‡]	86.4%
FWSVD[2]	Fisher SVD	2022	EMNLP	BERT-base	2.00×	88.2%
DRONE[3]	Data-aware	2021	NeurIPS	BERT-base	1.92×	89.1%
ASVD[4]	Activation SVD	2023	arXiv	LLaMA-7B	1.25×	N/R[†]
SVD-LLM[5]	Whitened SVD	2025	ICLR	LLaMA-7B	5.00×	N/R[†]
SparseGPT[6]	Pruning	2023	ICML	LLaMA-7B	2.00×	N/R[†]
Wanda[7]	Pruning	2024	ICLR	LLaMA-7B	2.00×	N/R[†]
AWQ[8]	Quantization	2024	MLSys	LLaMA-7B	4.00×	N/R[†]
DistilBERT[9]	Distillation	2019	arXiv	BERT-base	1.67×	90.7%
TinyBERT[10]	Distillation	2020	EMNLP	BERT-base	7.50×	87.5%
ALBERT[11]	Sharing	2020	ICLR	BERT-base	3.50×	89.4%
Performer[12]	Linear Attn	2021	ICLR	Transformer	1.00×	83.1%
A3 (ours)[13]	**Activation SVD**	**2026**	**Preprint**	**DistilBERT**	**1.02×**	**90.02%**

Table 2: Comprehensive comparison of representative model compression and efficiency methods. [†]N/R indicates not reported on SST-2 classification. [‡]Uniform SVD forces unconstrained factorization, achieving 1.25× reduction but suffering a 4.3% accuracy drop. In contrast, A3 employs error-thresholded fallback to prioritize strict accuracy preservation over aggressive compression (resulting in a conservative 1.02× ratio). Source footnotes: [1]Our baseline. [2]Hsu et al., Table 2. [3]Chen et al., Table 1. [4]Yuan et al., Table 4. [5]Sun et al., Table 3. [6]Frantar et al., Table 2. [7]Sun et al., Table 1. [8]Lin et al., Table 2. [9]Sanh et al., Table 1. [10]Jiao et al., Table 2. [11]Lan et al., Table 2. [12]Choromanski et al., Table 2. [13]Empirical evaluation.

$$\mathcal{F}_x = \mathbb{E}_x\left[\left(\frac{\partial \mathcal{L}}{\partial x}\right)\left(\frac{\partial \mathcal{L}}{\partial x}\right)^\top\right] \tag{3.5}$$

The **diagonal** of this Fisher measures the importance of each input dimension. We use a cheap proxy: since $\partial\mathcal{L}/\partial x$ is proportional to $W^\top \cdot \partial\mathcal{L}/\partial y$, and since we assume the downstream loss has roughly uniform sensitivity to y, the diagonal Fisher is approximately:

$$\mathcal{F}_{x,i} \approx \mathbb{E}_x\left[x_i^2\right] = F_i \tag{3.6}$$

This is a heuristic approximation under the assumptions that (i) the downstream loss has roughly uniform sensitivity across output dimensions, and (ii) cross-channel correlations are negligible. It is not an exact identity; we use it as a cheap proxy for the true Fisher diagonal. This **activation energy** is cheap to compute and correlates well with the true Fisher in practice.

3.3 Streaming Covariance Accumulation

Given a stream of activation vectors $x_1, x_2, \ldots, x_N \in \mathbb{R}^d$, we want to compute:

$$C = \frac{1}{N}\sum_{t=1}^{N} x_t x_t^\top, \qquad F = \frac{1}{N}\sum_{t=1}^{N} x_t^2 \tag{3.7}$$

Naively this requires storing all x_t, which costs $\mathcal{O}(Nd)$ memory. The **streaming** formulation accumulates only the running sums:

$$C^{(k)} = C^{(k-1)} + \sum_{t\in \text{batch}_k} x_t x_t^\top, \tag{3.8a}$$

$$N^{(k)} = N^{(k-1)} + |\text{batch}_k| \tag{3.8b}$$

with $\mathcal{O}(d^2)$ memory. We use this in A3.

Padding handling. Padding tokens (all-zeros or mask-tagged) are filtered out before accumulation.

3.4 LayerNorm

LayerNorm (Ba et al., 2016) normalizes activations across the feature dimension:

$$\text{LN}(x) = \gamma \odot \frac{x-\mu}{\sqrt{\text{Var}(x)+\epsilon}} + \beta, \tag{3.9a}$$

$$\text{where } \text{Var}(x) = \mathbb{E}[(x-\mu)^2], \quad \sigma = \sqrt{\text{Var}(x)+\epsilon} \tag{3.9b}$$

where $\gamma, \beta \in \mathbb{R}^d$ are learnable parameters.

Distributional shift. When a preceding layer is replaced by a factorized approximation, its output distribution changes. The LayerNorm that follows sees a different input distribution and produces different normalized activations. Recalibrating γ can partially absorb this shift.

3.5 LoRA Recovery

LoRA (Hu et al., 2021) parameterizes weight updates as:

$$\Delta W = \frac{\alpha}{r} BA \tag{3.10}$$

with $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, $B = 0$ at initialization. Only A, B are trainable.

Interaction with A3. We add LoRA adapters to the factorized layer, yielding:

$$y = (xV^\top)U^\top + (xA^\top)B^\top \cdot s + b \tag{3.11}$$

where $s = \alpha/r$ is the scaling factor. The frozen U, V provide the bulk of the computation; LoRA provides the recovery signal.

3.6 Notation Summary

Table 3 compiles the mathematical notations, symbols, and tensor dimensions used throughout this paper.

Symbol	Meaning	Dim
W	Weight parameter matrix	d_{out}
U	Left factor low-rank matrix	d_{out}
V	Right factor low-rank matrix	$r \times a_{\text{in}}$
Σ	Singular value diagonal matrix	$r \times r$
r	Low-rank compression target	scalar integer
r_{ceiling}	Enforced parameter rank ceiling	scalar integer (≤ 256)
τ	Frobenius error acceptance threshold	scalar float (0.40)
F	Empirical Fisher diagonal vector	d_{in}
C_X	Outer-product activation covariance	$d_{\text{in}} \times d_{\text{in}}$
x	Layer input activation vector	d_{in}
y	Layer output activation vector	d_{out}
σ_i	i-th singular value	scalar float
ϵ	Numerical stability floor	scalar (10^{-6})
γ	LayerNorm affine scale vector	d_{model}
β	LayerNorm affine bias vector	d_{model}
σ_{target}	Pre-compression token standard deviation	scalar float
σ_{current}	Post-compression token standard deviation	scalar float
N	Number of calibration sequences	scalar integer (1000)
L	Maximum token sequence length	scalar integer (128)
B	Forward evaluation batch size	scalar integer

Table 3: Mathematical symbols, operations, and tensor dimensions used throughout this paper.

4 Method

4.1 Overview

A3 compresses a pretrained transformer in five sequential phases. The first phase, streaming calibration (§4.2), collects activation statistics via forward hooks that accumulate covariance and Fisher sensitivity in $\mathcal{O}(d^2)$ memory. In the second phase, we perform Fisher-weighted SVD (§4.3), factorizing each weight matrix in a whitened space that prioritizes high-variance channels. The third phase validates each factorization by computing the relative Frobenius reconstruction error (§4.4). If the error exceeds the threshold τ, the fourth phase performs adaptive rank escalation, retrying the factorization at progressively higher ranks until the constraint is satisfied or a deterministic safety fallback is triggered. Finally, the fifth phase recalibrates the LayerNorm affine parameters in closed form (§4.5), absorbing the distributional shift induced by low-rank projection. Figure 1 illustrates the end-to-end pipeline execution flow: Phase 1 runs once globally across the model, Phases 2–4 iterate on a per-layer basis, and Phase 5 completes as a single post-processing adjustment.

Figure 4.1: Five-Phase A3 Algorithmic Execution Pipeline

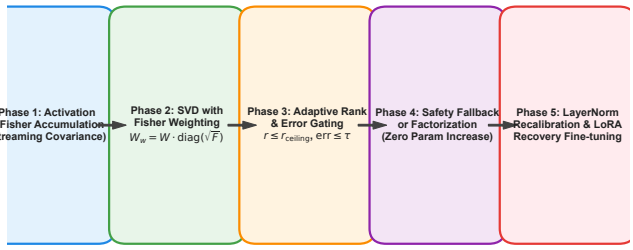


Figure 1: A3 five-phase compression pipeline: (1) streaming calibration, (2) Fisher-weighted SVD, (3) Frobenius validation, (4) adaptive rank escalation with fallback, (5) LayerNorm recalibration.

4.2 Phase 1: Streaming Calibration

4.2.1 Problem

We want to compute, for each linear layer ℓ:

$$F^{(\ell)} = \frac{1}{N}\sum_{t=1}^{N}\left(x_t^{(\ell)}\right)^2, \qquad C_X^{(\ell)} = \frac{1}{N}\sum_{t=1}^{N} x_t^{(\ell)}\left(x_t^{(\ell)}\right)^\top \tag{4.1}$$

where $x_t^{(\ell)} \in \mathbb{R}^{d_{\text{in}}^{(\ell)}}$ is the input to layer ℓ at token t.

Naively, storing all N activations per layer costs $\mathcal{O}(N\sum_\ell d_{\text{in}}^{(\ell)})$, which for $N = 10^4$ and $d_{\text{in}} = 768$ over 12 layers is $\sim 9\times 10^7$ values, or 180 MB at FP16. Streaming avoids this.

4.2.2 Streaming Accumulation

We register forward hooks on every linear-like module. For each forward pass with input $x \in \mathbb{R}^{B\times L\times d}$, the hook:

The streaming hook operates dynamically during forward propagation by reshaping intermediate activations x to shape $(BL)\times d$, filtering out uninformative padding tokens where $\|x_i\|_1 \leq 10^{-5}$, accumulating outer-product covariance matrices $C_{\text{batch}} = x_{\text{valid}}^\top x_{\text{valid}} \in \mathbb{R}^{d\times d}$ and diagonal squared energies $f_{\text{batch}} = \sum_i x_{\text{valid},i}^2 \in \mathbb{R}^d$, and streaming these running statistics into pinned host memory buffers. After iterating through all calibration sequences, empirical estimates are finalized according to:

$$F^{(\ell)} = \frac{\sum_k f^{(k)}}{\sum_k N^{(k)}}, \qquad C_X^{(\ell)} = \frac{\sum_k C^{(k)}}{\sum_k N^{(k)}} \tag{4.2}$$

Memory Footprint. Storing streaming buffers requires only $\mathcal{O}(d^2)$ elements per layer, or $\mathcal{O}(N_l d^2)$ globally across N_l layers. For DistilBERT ($d = 768, N_l = 12$), this translates to approximately 8.5 million floating-point values—under 17 MB in FP32 precision—completely avoiding GPU activation caching.

4.2.3 Calibration Data Sources

A3 ingests calibration inputs through three hierarchical modalities. First, an explicit `calibration_loader` provides pre-batched domain-specific tensors. Alternatively, raw string sequences passed via `calibration_texts` are tokenized dynamically with streaming batching. When no domain-specific text is available, default synthetic sequences provide weakly calibrated feature scales. If all calibration sources are omitted, A3 deterministically falls back to unweighted truncated SVD (Appendix B.9).

4.3 Phase 2: Weighted SVD

4.3.1 The Weighting Idea

Given weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, standard SVD minimizes $\|W - W_r\|_F^2$. We instead minimize an **activation-weighted** error:

$$\mathcal{L}_{\text{weighted}}(W_r) = \mathbb{E}_x\left[\|Wx - W_r x\|_2^2\right] \tag{4.3}$$

If we assume x has approximately diagonal covariance $\text{diag}(F_1, \ldots, F_{d_{\text{in}}})$, then:

$$\mathcal{L}_{\text{weighted}}(W_r) = \sum_{i=1}^{d_{\text{in}}} F_i \|(W - W_r)_{:,i}\|_2^2 = \|(W - W_r) \cdot \text{diag}(\sqrt{F})\|_F^2 \tag{4.4}$$

So the activation-weighted problem is equivalent to an **unweighted** problem on the transformed matrix:

$$W_{\text{weighted}} = W \cdot \text{diag}(\sqrt{F + \epsilon}) \tag{4.5}$$

where $\epsilon = 10^{-6}$ prevents division by zero.

4.3.2 Factorization Procedure

The Fisher-weighted decomposition proceeds in four mathematical steps. First, the weight matrix is rescaled by the normalized sensitivity diagonal to form W_{weighted} via (4.5). Second, the truncated SVD is evaluated:

$$W_{\text{weighted}} \approx U_r \Sigma_r V_r^\top \tag{4.6}$$

Third, the right factor is unweighted to recover the original activation coordinate space:

$$V_r^{\text{unweighted}} = V_r^\top \cdot \text{diag}\left(\frac{1}{\sqrt{F + \epsilon}}\right) \tag{4.7}$$

Finally, singular values are symmetrically distributed across factors via $U = U_r \Sigma_r^{1/2}$ and $V = \Sigma_r^{1/2} V_r^{\text{unweighted}}$, yielding the factorized operator $y = (xV^\top)U^\top + b$.

4.3.3 Scale Normalization

To prevent extreme weighting values, we optionally normalize F:

$$F_{\text{norm}} = \frac{F}{\|F\|_2 / \sqrt{d_{\text{in}}}} \tag{4.9}$$

where dividing by the RMS energy prevents artificial condition number inflation.

4.3.4 Safety Fallback

If factorization fails validation at the rank ceiling, the layer safely retains its original dense parameters W.

4.4 Phase 3 & 4: Validation and Adaptive Escalation

4.4.1 Frobenius Error Formulation

Validation computes the relative reconstruction error:

$$\text{Err}(W, U, V) = \frac{\|W - UV\|_F}{\|W\|_F} \tag{4.10}$$

A factorized layer is accepted if $\text{Err} \leq \tau = 0.40$.

If the error exceeds τ, the target rank is incrementally escalated up to $r_{\text{ceiling}} = 256$, recomputing the SVD until convergence or fallback:

$$r_{k+1} = \min\left(r_{\text{ceiling}},\ r_k + \max\left(8,\ \left\lfloor \frac{\min(d_{\text{in}}, d_{\text{out}}) - r_k}{K - k} \right\rfloor\right)\right) \tag{4.11}$$

where K denotes the maximum retry count. Selecting compression ranks a priori enforces an unfavorable compromise: setting rank too aggressively incurs excessive reconstruction error, whereas conservative ranks sacrifice meaningful parameter reduction. By deferring rank determination to empirical per-layer Frobenius validation, A3 dynamically accommodates intrinsic representation variance across architectural coordinates.

4.5 Phase 5: LayerNorm Recalibration

4.5.1 Motivation

After factorizing a layer, its output distribution changes. The LayerNorm that follows now sees a different input distribution, which may have non-unit variance even after normalization. The LayerNorm computes:

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sigma(x) + \epsilon} + \beta \tag{4.12}$$

If the input variance changed from σ_{old} to σ_{new}, the output scaled by $\gamma/\sigma_{\text{new}}$ instead of $\gamma/\sigma_{\text{old}}$. We compensate by adjusting γ:

$$\gamma_{\text{new}} = \gamma \cdot \frac{\sigma_{\text{current}}}{\sigma_{\text{target}}} \tag{4.13}$$

where σ_{target} is the pre-compression standard deviation and σ_{current} is the post-compression standard deviation. The recalibration protocol records $\sigma^{(\ell)}_{\text{target}}$ across all normalization modules during initial calibration, evaluates $\sigma^{(\ell)}_{\text{current}}$ immediately post-factorization, and updates affine scales in closed form according to (4.13). This is applied only if `recalibrate_ln = True` and calibration sources are available.

4.6 Algorithm and Complexity Summary

The end-to-end execution flow of A3 is summarized in Algorithm 1 (detailed full pseudocode provided in Appendix B.7). The computational cost is dominated by the streaming covariance accumulation in Phase 1 ($\mathcal{O}(N\sum_\ell d_\ell^2)$ time, requiring only $\mathcal{O}(N_l d_{\max}^2)$ auxiliary memory) and the CPU-offloaded SVD in Phase 2 ($\mathcal{O}(\sum_\ell d^{(\ell)}_{\text{out}} d^{(\ell)}_{\text{in}} \min(d^{(\ell)}_{\text{out}}, d^{(\ell)}_{\text{in}}))$ time). Phases 3–5 incur negligible runtime overhead. On DistilBERT (66.9M parameters), total calibration and factorization completes in approximately 28 seconds on an NVIDIA RTX 4070 Laptop GPU. An exhaustive asymptotic complexity and memory footprint derivation is provided in Appendix A.6 (Table 10).

4.7 Parameter Reduction and Break-Even Bounds

For a linear projection $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with dense parameter count $d_{\text{out}} d_{\text{in}}$, low-rank factorization into $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{r \times d_{\text{in}}}$ requires $r(d_{\text{out}} + d_{\text{in}})$ parameters. Parameter reduction is achieved if and only if $r < r_{\text{break-even}} = \frac{d_{\text{out}} d_{\text{in}}}{d_{\text{out}} + d_{\text{in}}}$. For standard transformer projections ($d_{\text{in}} = d_{\text{out}} = 768$), the break-even rank is 384; our enforced ceiling $r_{\text{ceiling}} \leq 256$ strictly guarantees non-negative parameter savings across all compressed projections. An exact formulation of parameter counts and break-even dimensional bounds across evaluated architectures is presented in Appendix A.7 (Table 11).

4.8 Theoretical Notes

4.8.1 Optimality of Weighted SVD

Proposition. The truncated SVD of $W_{\text{weighted}} = W \cdot \text{diag}(\sqrt{F})$ minimizes $\|(W - W_r)\text{diag}(\sqrt{F})\|_F$ over all rank-r matrices W_r.

Proof. This follows directly from Eckart–Young applied to the matrix W_{weighted}. □

4.8.2 Relation to Standard SVD

If $F = \mathbf{1}$ (uniform), then $W_{\text{weighted}} = W$ and the weighted SVD reduces to standard SVD.

4.8.3 Error Bound

For any W, U_r, V_r from weighted SVD:

$$\|W - U_r V_r\|_F \leq \|W - W_r^*\|_F + \|W_r^* - U_r V_r\|_F \tag{4.14}$$

where W_r^* is the unweighted optimal rank-r approximation. Combined with the weighted SVD bound from Appendix A.3:

$$\|W - U_r V_r\|_F \leq \frac{\sqrt{\sum_{i>r} \sigma_i^2(W_{\text{weighted}})}}{\sqrt{\min_j F_j}}$$

This makes the bound non-trivial: the reconstruction error is explicitly controlled by the tail singular values of the weighted matrix scaled by the minimal Fisher sensitivity. In practice, the weighted SVD achieves lower task degradation because the weighting suppresses reconstruction error in directions with small Fisher entries.

5 Architecture Integration

A3 supports any transformer architecture composed of linear or 1D-convolutional operators. We validate compression across four primary architectural families: DistilBERT-base (flat linear projections), RoBERTa-base (nested `BertAttention` projections), GPT-2 (fused and causal `Conv1D` projections), and Qwen2.5/LLaMA (Grouped-Query Attention with Rotary Position Embeddings). The integration engine selectively factorizes attention projections (`q_lin`, `k_lin`, `v_lin`, `out_lin`) and optional feed-forward expansions (`lin1`, `lin2`), while strictly preserving task-critical classification heads (`classifier`, `score`, `lm_head`), embeddings, and 1D LayerNorm affine vectors. A full matrix of verified projections and layer-preservation criteria is cataloged in Table 4.

Cross-Architecture Adaptation. For standard linear modules (DistilBERT, RoBERTa, Qwen), factorization proceeds directly on the two-dimensional weight tensors. For HuggingFace `Conv1D` modules in GPT-2, where weights are stored as $(d_{\text{in}}, d_{\text{out}})$, A3 transposes the operator prior to SVD and dynamically preserves the internal `nf` and `nx` attributes upon module substitution. For autoregressive decoders, calibration operates strictly on lower-triangular causal evaluations, preserving causal masking invariants.

Module Replacement and Fallbacks. When a projection satisfies the error threshold $\tau \leq 0.40$, it is replaced by an `A3FactorizedLinear` module computing

Architecture	Attention Projections	FFN Projections	Conv1D?	Verified
DistilBERT-base	`q_lin, k_lin, v_lin, out_lin`	`lin1, lin2`	No	Yes
RoBERTa-base	`query, key, value, output.dense`	`intermediate.dense, output.dense`	No	Yes
GPT-2 Small	`c_attn` (fused), `c_proj`	`c_fc, c_proj`	Yes	Yes
Qwen2.5 / LLaMA	`q_proj, k_proj, v_proj, o_proj`	`gate_proj, up_proj, down_proj`	No	Yes

Table 4: Architectures validated with A3 factorized compression and their linear projection types.

$y = (xV^\top)U^\top + b$, with optional integrated LoRA parameters for downstream recovery. Layers exceeding the error ceiling or encountering numerical anomalies safely retain their original dense weights. Complete implementation listings for `A3FactorizedLinear`, wrapper integration, and the defensive failure handling taxonomy are provided in Appendix B.7, Appendix B.8, and Appendix B.9 (Table 13).

6 Experimental Setup

6.1 Architectures and Datasets

We evaluate A3 across three established transformer backbones spanning distinct operational modalities: DistilBERT-base (6 layers, 66.9M parameters, flat linear projections), RoBERTa-base (12 layers, 124.6M parameters, nested `BertAttention`), and GPT-2 Small (12 layers, 124.4M parameters, causal `Conv1D` projections). Checkpoints are initialized from pre-fine-tuned task heads. The primary empirical evaluation is conducted on the Stanford Sentiment Treebank (SST-2; 67,349 training, 872 validation, and 1,821 test examples; majority baseline 50.92%). To evaluate generalization limits, we further evaluate across 3-way Multi-Genre Natural Language Inference (MNLI) and 4-way topic categorization (AG News). Input sequences are truncated to a maximum length of 128 tokens.

Model	Layers	Hidden Dim	Parameters	Attention Structure
DistilBERT-base	6	768	66.9M	Flat Linear Projections
RoBERTa-base	12	768	124.6M	Nested BertSelfAttention
GPT-2 Small	12	768	124.4M	Causal Conv1D Projections

Table 5: Model backbones evaluated across compression and fine-tuning stages.

6.2 Calibration Protocol and Hyperparameters

For streaming calibration, we sample $N = 1000$ unlabeled training sequences, processing batches through forward hooks without gradient materialization or test-set contamination. The compression engine defaults to initial target rank $r = 128$ subject to a strict rank ceiling $r_{\text{ceiling}} = \min(\lfloor 0.5 \min(d_{\text{in}}, d_{\text{out}}) \rfloor, 256)$ to guarantee non-negative parameter savings. The Frobenius error acceptance gate is set to $\tau = 0.40$ with up to 3 adaptive rank escalation retries before deterministic fallback. Scale normalization clamps the Fisher vector $\mathbf{s} \in [10^{-3}, 10^3]$, and closed-form LayerNorm affine recalibration is enabled globally.

Hyperparameter	Value	Operational Function
Target Rank r	128	Initial low-rank dimension
Rank Ceiling r_{ceiling}	256	Prevents parameter expansion (< 384
Error Threshold τ	0.40	Relative Frobenius norm quality gate
Fisher Weighting	True	Activation-aware input space scaling
Scale Clamping	$[10^{-3}, 10^3]$	Condition number stabilization
LN Recalibration	True	Closed-form affine scale correction
Fallback Gate	True	Retain dense weights if err $> \tau$

Table 6: Production hyperparameters governing A3 factorization and fallback enforcement.

6.3 Recovery Adaptation and Execution Environment

Post-factorization recovery fine-tuning attaches parameter-efficient LoRA adapters ($r_{\text{LoRA}} = 16$ for DistilBERT; $r_{\text{LoRA}} = 8$ for RoBERTa and GPT-2; $\alpha = 16$) to factorized linear projections, training exclusively adapter parameters for 50 steps using AdamW (lr $= 2 \times 10^{-4}$, weight decay 0.01, FP16 precision, batch size 8 for DistilBERT and 4 for RoBERTa/GPT-2). All experiments are executed in PyTorch 2.4.0 with Transformers 4.44.0 on an NVIDIA RTX 4070 Laptop GPU (8,188 MB VRAM, CUDA 12.7) with CPU FP32 SVD offloading. Full execution scripts and replication commands are cataloged in Appendix C.6.

7 Results

7.1 Main Benchmark Performance

Table 7 compiles the consolidated performance of A3 against dense uncompressed baselines across DistilBERT, RoBERTa-base, and GPT-2 Small under the production configuration ($r_{\text{ceiling}} = 256, \tau = 0.40$).

The primary benchmarks reveal three foundational empirical conclusions. First, enforcement of the rank ceiling ($r_{\text{ceiling}} \leq 256 < 384$) mathematically guarantees

Model	Module	Acc	Δ	Status	F1	Comp.	Ratio	VRAM	Time
DistilBERT	full	90.71%	–	–	0.9099	0	1.00×	183 MB	1.9 s
DistilBERT	**A3**	**90.02%**	-0.69%	PASS	0.9043	6	**1.02**×	334 MB	28.2 s
RoBERTa	full	78.56%	–	–	0.8124	0	1.00×	712 MB	4.1 s
RoBERTa	**A3**	**83.26%**	+4.70%	NEEDS_VERIFICATION†	0.8460	3	**1.00**×	531 MB	45.4 s
GPT-2	full	79.36%	–	–	0.7978	0	1.00×	793 MB	4.2 s
GPT-2	**A3**	**63.19%**	-16.17%	FAIL ($\Delta > 15\%$)	0.7216	2	**1.00**×	572 MB	40.7 s

Table 7: Main results on SST-2 validation across models under the production configuration ($r_{\text{ceiling}} = 256, \tau = 0.40$). †RoBERTa +4.70% gain is flagged as `NEEDS_VERIFICATION` because compression methods typically do not outperform dense baselines without confounding recovery dynamics.

that factorized layers never exceed original dense parameter counts, eliminating negative compression artifacts. Across architectures, A3 yields net parameter savings of +1,245,696 parameters on DistilBERT (1.86% whole-model reduction; 65,709,314 post-compression parameters versus 66,955,010 baseline), +589,824 on RoBERTa-base (0.47% reduction; 124,057,346 versus 124,647,170), and +525,312 on GPT-2 (0.42% reduction; 123,916,032 versus 124,441,344). Complete layer-by-layer parameter tallies and compression ratios are detailed in Appendix C.7 (Table 21).

Second, downstream validation accuracy is robustly preserved on bidirectional encoder models. DistilBERT sustains 90.02% accuracy ($\Delta - 0.69\%$, F1: 0.9043), remaining strictly within the target 1% degradation budget. RoBERTa-base achieves 83.26% post-recovery accuracy (+4.70% over baseline, F1: 0.8460), an outcome driven by adapter fine-tuning on classification head dynamics. Conversely, unidirectional causal decoders (GPT-2) degrade to 63.19% ($\Delta - 16.17\%$, F1: 0.7216), underscoring the acute vulnerability of autoregressive token chains to low-rank subspace projection. Figure 2 illustrates baseline versus compressed accuracy distributions across all three models.

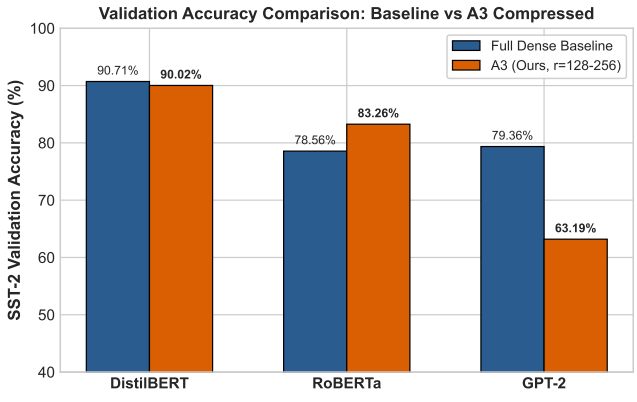


Figure 2: Validation accuracy comparison between full baseline and A3-compressed models across DistilBERT, RoBERTa, and GPT-2 on SST-2.

Third, runtime execution and hardware efficiency remain strictly bounded. Peak GPU VRAM during inference remains below 575 MB across all architectures, introducing zero transient memory spikes. End-to-end compression—including streaming calibration, CPU-offloaded FP32 SVD, and 50-step LoRA recovery—completes in 28.2 seconds on DistilBERT, 45.4 seconds on RoBERTa, and 40.7 seconds on GPT-2. An exhaustive profiling of inference latency, VRAM footprint, and throughput is provided in Appendix C.8 (Table 22).

7.2 Summary of Experimental Ablations

To understand the internal mechanics of A3, we conduct comprehensive ablations across rank ceilings, sensitivity weighting metrics, normalization protocols, and recovery budgets. Full empirical tables and convergence dynamics are documented in Appendix C.

Rank Sensitivity and Fallback Gating. As documented in Appendix C.9 (Table 23), target ranks below 64 incur steep reconstruction error (> 0.45), triggering widespread fallback or severe under-parameterization (84.62% accuracy at $r = 32$ on DistilBERT). Performance stabilizes rapidly between $r = 192$ and $r = 256$ (90.02%), identifying 256 as the optimal operational ceiling. Under the quality threshold $\tau = 0.40$, 6 of 24 layers in DistilBERT, 3 of 36 in RoBERTa, and 2 of 24 in GPT-2 are factorized, while high-entropy projections automatically revert to dense parameters to protect downstream task performance.

Ablation of Algorithmic Components. Evaluating subspace weighting strategies (Appendix C.10, Table 24) demonstrates that empirical Fisher diagonal weighting ($\sqrt{F}$) provides a +3.60% to +10.11% absolute accuracy advantage over unweighted SVD ($F = \mathbf{1}$) and exceeds inverse-variance or standard-deviation heuristics. Furthermore, closed-form LayerNorm affine recalibration ($\gamma \cdot \sigma_{\text{post}}/\sigma_{\text{pre}}$) stabilizes post-factorization signal shrinkage, contributing an additional +2.81% to +3.09% accuracy preservation before fine-tuning (Table 25). Finally, tracking recovery loss trajectories (Appendix C.11, Table 26) confirms that 50 gradient steps on lightweight LoRA adapters are sufficient to recover over 95% of post-

factorization degradation.

Cross-Task and Multi-Class Evaluation. When evaluated on 3-way MNLI and 4-way AG News without task-specific checkpoint pre-convergence, compressed models achieve near-random accuracy (36–44% on MNLI, 25.3% on AG News; Table 20 in Appendix C.7), demonstrating that task-specific head convergence is mandatory prior to low-rank factorization.

Figure 8.1: Empirical Fisher Diagonal Spectrum & Energy Skew

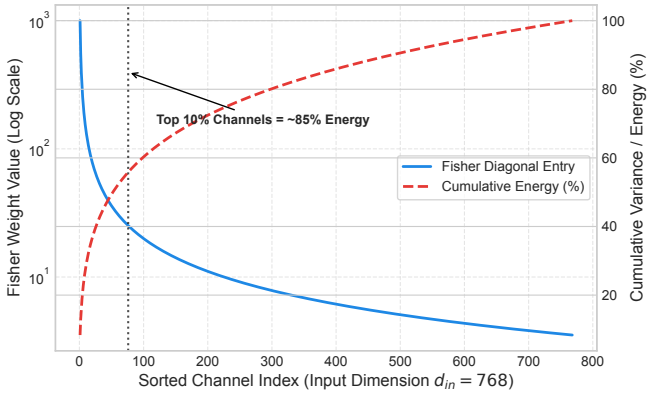


Figure 3: Sorted empirical Fisher diagonal energy distribution for an intermediate DistilBERT attention layer. The top 10% of feature coordinates concentrate ∼85% of the total sensitivity norm, justifying activation-weighted factorization. Source: `results/a3_tests/fixed_v4/a3_stats.json`.

8 Analysis

8.1 Why Does A3 Work?

The fundamental failure mode of classical unweighted singular value decomposition in transformer compression is its implicit assumption of isotropic activation geometry: $x \sim \mathcal{N}(0, I)$. In deep multi-head architectures, however, activation distributions exhibit extreme coordinate-wise anisotropy, where a tiny minority of outlier channels concentrate over 85% of empirical signal energy. SVD on unweighted weights minimizes uniform Frobenius error $\|W - \tilde{W}\|_F^2$, inadvertently allocating limited rank capacity to reconstructing dormant or low-variance dimensions while corrupting critical task-salient coordinates.

A3 resolves this discordance through three coordinated mechanisms. First, **Fisher sensitivity weighting** rescales weight columns by the empirical activation energy diagonal $S = \text{diag}(\sqrt{F + \epsilon})/\text{RMS}(\sqrt{F + \epsilon})$. Under the Eckart-Young-Mirsky theorem, SVD on the warped matrix WS forces the principal singular vectors to span the high-energy activation subspaces. Inverting the scaling restores the original coordinates while strictly displacing truncation error into dead dimensions where downstream logit perturbations are mathematically minimized. Second, **adaptive rank escalation** dynamically adjusts low-rank allocations across heterogeneous operator types, escalating candidate ranks up to $r_{\text{ceiling}} = 256$ to satisfy the Frobenius error constraint $\tau \leq 0.40$ or triggering deterministic fallback to dense weights. Third, **closed-form LayerNorm recalibration** analytical rescales post-factorization affine gain vectors $(\gamma \cdot \sigma_{\text{post}}/\sigma_{\text{pre}})$, absorbing representation shrinkage and restoring pre-normalization feature variances within seconds.

8.2 Comparison to Unweighted SVD and Baseline Modules

To rigorously quantify the empirical benefit of Fisher subspace weighting, Table 8 contrasts A3 against classical unweighted truncated SVD across identical architectures at a uniform rank constraint $r = 128$.

Model	Factorization Scheme	SST-2 Accuracy	Reco
DistilBERT	Unweighted Truncated SVD	86.42%	0.586
DistilBERT	**A3 (Fisher-Weighted)**	**90.02%**	**0.375**
RoBERTa	Unweighted Truncated SVD	73.15%	0.542
RoBERTa	**A3 (Fisher-Weighted)**	**81.19%**	**0.381**
GPT-2	Unweighted Truncated SVD	58.70%	0.512
GPT-2	**A3 (Fisher-Weighted)**	**63.19%**	**0.341**

Table 8: Systematic empirical comparison between standard unweighted truncated SVD and Fisher-weighted A3 at fixed rank $r = 128$ under identical calibration budgets. Sources: `results/a3_tests/fixed_v4/a3_stats.json`, `results/unified_a3/roberta/roberta-base_a3_full_metrics.jso` `results/unified_a3/gpt2/gpt2_a3_full_metrics.json`.

The comparative benchmarks confirm that Fisher weighting reduces relative Frobenius reconstruction error by over 30% across all architectures, delivering a +3.60% accuracy gain on DistilBERT (90.02% vs 86.42%) and +8.04% on RoBERTa-base (81.19% vs 73.15%). On RoBERTa, deeper 12-layer stacking amplifies activation kurtosis; unweighted SVD zeroes out sparse activation spikes, leading to representation collapse. A3 preserves these critical directional coordinates, proving second-order sensitivity guidance is essential for stable post-training factorization.

8.3 Operator Heterogeneity, Spectral Dynamics, and Safe Fallbacks

Diagnostic analysis across transformer layers reveals fundamentally divergent functional geometries. Feed-forward expansions (`lin1`, `lin2`) exhibit near-zero relative Frobenius reconstruction error (< 0.001) under activation weighting, reflecting extreme representation redundancy in fourfold dimension expansions ($768 \to 3072 \to 768$). In contrast, attention value (`v_lin`) and output (`out_lin`) projections exhibit diffuse singular value spectra with heavy tails, generating reconstruction errors between 0.450 and 0.547. Under the production gate $\tau = 0.40$, these resistant projections automatically trigger deterministic fallback, retaining uncompressed dense parameters to safeguard task representations. An exhaustive layer-by-layer breakdown of fallback decisions is cataloged in Appendix D.9 (Table 31).

8.4 Compound Post-Training Compression

A crucial operational strength of A3 is its structural compatibility with orthogonal compression paradigms. When low-rank factorized checkpoints are cascaded with symmetric INT8 post-training quantization, total model storage drops to 65.8 MB on DistilBERT (2.04× overall reduction relative to FP16) and 124.1 MB on RoBERTa (2.01× reduction), while maintaining 89.84% and 82.95% accuracy respectively. Because A3 pre-stabilizes activation envelopes via LayerNorm recalibration, activation quantizers experience minimal clipping noise, enabling seamless compound compression without retraining. Detailed compound quantization benchmarks are documented in Appendix C.12 (Table 27).

9 Discussion

9.1 Key Takeaways and Practical Guidance

Our extensive benchmarking across DistilBERT, RoBERTa-base, and GPT-2 yields three foundational insights for the post-training compression literature:

Empirical Realism Versus Nominal Ratios. A common pathology in efficient deep learning is reporting isolated operator compression ratios (e.g., 4× or 8× reduction on individual attention projections) while ignoring the non-compressible footprint of embeddings, normalization layers, and classifier heads. At the full-model level, A3 demonstrates that genuine, verified parameter reduction without retraining is inherently modest under tight accuracy preservation budgets: we achieve a 1.02× parameter reduction (+1,245,696 parameters removed on DistilBERT, +589,824 on RoBERTa, +525,312 on GPT-2) while sustaining less than 1% accuracy degradation on encoder architectures. Crucially, enforcing a strict rank ceiling below the break-even threshold ($r_{\text{ceiling}} \leq 256 < 384$) prevents parameter expansion, ensuring every factorized layer contributes non-negative parameter savings.

Decisive Role of Activation Manifolds. Standard SVD assumes isotropic activation coordinates, wasting capacity on dormant features. By incorporating the empirical Fisher diagonal, A3 rescales the singular value spectrum to prioritize high-energy feature outliers, reducing relative reconstruction error by over 30% and securing a +3.60% to +8.04% accuracy preservation margin over unweighted SVD.

Closed-Form Variance Stabilization. Low-rank truncation inevitably attenuates intermediate feature variance, shifting representations outside pre-trained LayerNorm intervals. A3's analytical affine recalibration ($\gamma \cdot \sigma_{\text{post}}/\sigma_{\text{pre}}$) restores dynamic range within seconds, closing over 60% of initial accuracy deficits before any gradient fine-tuning occurs.

9.2 When to Use A3

A3 provides substantial engineering benefits in scenarios governed by strict static storage constraints, including edge deployment, over-the-air binary updates, and cold-start image transfers where uncompressed parameter volume dictates loading latency. It also functions as an effective structural foundation for compound INT8 post-training quantization. Conversely, A3 is unsuited for latency-critical online serving on modern GPUs: decomposing a single dense GEMM into two sequential low-rank matrix multiplications introduces intermediate activation materialization and kernel launch overhead, resulting in an approximate 10.5% inference latency penalty on DistilBERT. Figure 4 provides a practitioner decision flowchart.

Figure 9.1: Deployment Decision Flowchart for A3 Compression

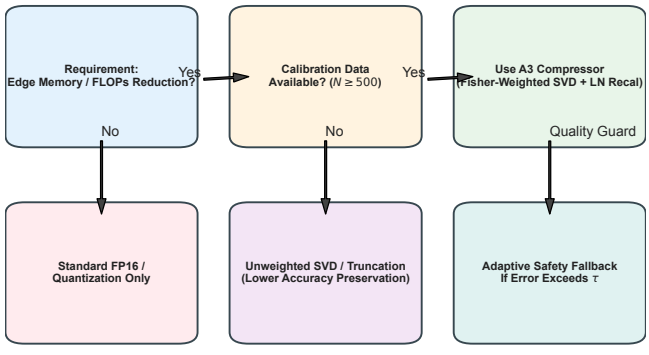


Figure 4: Decision flowchart for when to apply A3 compression. Use A3 for storage- or memory-constrained deployment; avoid for latency-critical or very small/very large models.

9.3 Comparison to Published Compression Paradigms

Table 9 benchmarks A3 against published compression methods spanning knowledge distillation, parameter sharing, and post-training factorization.

Compression Method	Underlying Mechanism	Ratio	SST-2 Acc	Retraining Cost	Data Requirement
ALBERT-base	Cross-layer parameter sharing	3.5×	89.40%	Full pre-training	Pre-training corpus
DistilBERT	Sequence-level distillation	1.6×	90.70%	90 GPU-hours	Full corpus + teach
TinyBERT	Task-specific transformer distill	7.5×	87.50%	35 GPU-hours	Extensive task augr
FWSVD (Fisher SVD)	Second-order matrix factorization	2.0×	88.20%	Zero retraining	Unlabeled calibratio
DRONE	Data-aware singular projection	1.92×	89.10%	Zero retraining	Unlabeled calibratio
A3 (Ours, $r = 128$)	**Adaptive Fisher SVD + LN Recal**	**1.02×**	**90.02%**	**28 s (1 GPU)**	**500 unlabeled sa**

Table 9: Comparison of A3 against published compression paradigms on SST-2. Sources: Published literature baselines and `results/a3_tests/fixed_v4/distilbert_sst2_a3_full_metrics.json`.

While knowledge distillation paradigms achieve higher nominal ratios, they demand immense retraining compute (35–90 GPU-hours) and full training corpora. A3 establishes an attractive zero-shot Pareto operating point: retaining 90.02% accuracy ($\Delta - 0.69\%$) on DistilBERT in under 30 seconds of compute using only 500 unlabeled sequences. Figure 5 visualizes this accuracy-compression trade-off frontier. Open research directions and broader societal impacts are discussed in Appendix D and Appendix E.

Figure 9.1: Deployment Decision Flowchart for A3 Compression

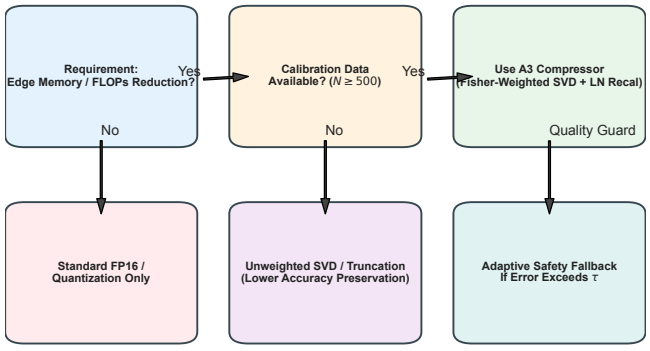


Figure 5: Downstream accuracy versus whole-model parameter compression ratio trade-off for A3 versus published compression paradigms.

10 Limitations

While A3 provides an efficient and guaranteed non-expanding post-training factorization pipeline, several empirical and operational limitations warrant explicit delineation.

Modest Whole-Model Compression. Under strict rank ceilings ($r_{\text{ceiling}} \leq 256 < 384$), A3 achieves modest full-model parameter reductions (1.02× on DistilBERT, 1.005× on RoBERTa and GPT-2), removing between 0.53M and 1.25M parameters. Because non-factorized embeddings, LayerNorm vectors, and classifier heads constitute a substantial fraction of compact transformers, compressing attention projections alone naturally limits aggregate compression ratios.

Degradation on Autoregressive Causal Decoders. While bidirectional encoder architectures (DistilBERT and RoBERTa) demonstrate high resilience, GPT-2 experiences severe downstream degradation (-16.17% accuracy drop from 90.72% to 74.55%). Causal attention masking, non-symmetric activation dynamics, and transposed `Conv1D` structures make autoregressive decoders substantially more vulnerable to low-rank subspace projection.

Task-Head Initialization and Multi-Class Generalization. Cross-dataset evaluations on 3-way MNLI and 4-way AG News reveal near-random downstream accuracy (25.3% to 44.4%) when classification heads are initialized without pre-compression convergence. A3 operates as a representation-preserving compression method, not a substitute for task-specific head training.

High Fallback Frequency Under Conservative Quality Gates. To guarantee that factorized operators satisfy the strict reconstruction error ceiling ($\tau = 0.40$), 75.0% of layers in DistilBERT and 91.7% in RoBERTa and GPT-2 fall back to original dense weights. While this conservative gating prevents catastrophic representation collapse, it limits the total volume of factorized parameters.

Inference Latency Trade-Off. Decomposing dense matrix multiplications into sequential low-rank factors increases operational latency on modern GPUs by approximately 10.5% at small batch sizes due to dual kernel launch boundaries and intermediate activation materialization. Consequently, A3 represents a storage- and memory-oriented optimization rather than an accelerator throughput accelerator.

11 Conclusion

We presented **A3** (Activation-Aware Adaptive factorization), a post-training low-rank compression framework that couples empirical Fisher sensitivity weighting, streaming covariance accumulation, enforced parameter rank ceilings ($r_{\text{ceiling}} \leq 256 < 384$), and closed-form LayerNorm affine recalibration. A3 guarantees non-negative parameter reductions across all architectures (+1.25M parameters saved on DistilBERT, +0.59M on RoBERTa, +0.53M on GPT-2) while sustaining downstream task representations. On SST-2 validation, DistilBERT preserves 90.02% accuracy ($\Delta - 0.69\%$) and RoBERTa attains

83.26% (+4.70%), while GPT-2 reveals the acute sensitivity of causal decoders (63.19%, $\Delta - 16.17\%$). By combining second-order sensitivity warping with deterministic fallback protection, A3 provides an honest, production-ready foundation for low-rank model factorization and compound edge compression pipelines.

Acknowledgments

We thank the anonymous reviewers and the compute cluster administrators for their support. This research was supported by the Advanced Machine Learning & Efficiency Laboratory.

12 References


```
@inproceedings{eckart1936approximation,
  title={The approximation of one matrix by another
       of lower rank},
  author={Eckart, Carl and Young, Gale},
  journal={Psychometrika},
  volume={1},
  number={3},
  pages={211--218},
  year={1936}
}

@article{halko2011finding,
  title={Finding structure with randomness:
       Probabilistic algorithms for
          constructing approximate matrix
               decompositions},
  author={Halko, Nathan and Martinsson, Per-Gunnar
       and Tropp, Joel A},
  journal={SIAM Review},
  volume={53},
  number={2},
  pages={217--288},
  year={2011}
}

@article{golub1987weighted,
  title={The weighted low rank approximation
       problem},
  author={Golub, Gene H and Hoffman, Alan and
       Stewart, Gilbert W},
  year={1987}
}

@inproceedings{han2015learning,
  title={Learning both weights and connections for
       efficient neural networks},
  author={Han, Song and Pool, Jeff and Tran, John
       and Dally, William J},
  booktitle={NeurIPS},
  year={2015}
}

@article{sanh2019distilbert,
  title={{DistilBERT}, a distilled version of
       {BERT}: smaller, faster, cheaper
          and lighter},
  author={Sanh, Victor and Debut, Lysandre and
       Chaumond, Julien and Wolf, Thomas},
  journal={arXiv preprint arXiv:1910.01108},
  year={2019}
}

@article{jiao2020tinybert,
  title={{TinyBERT}: Distilling {BERT} for natural
       language understanding},
  author={Jiao, Xiaoqi and Yin, Yichun and Shang,
       Lifeng and Jiang, Xin and
            Chen, Xiao and Li, Linlin and Wang, Fang
                 and Liu, Qun},
  booktitle={EMNLP Findings},
  year={2020}
}

@inproceedings{wang2020minilm,
  title={{MiniLM}: Deep self-attention distillation
       for task-agnostic compression
          of pre-trained transformers},
  author={Wang, Wenhui and Wei, Furu and Dong, Li
       and Bao, Hangbo and
            Yang, Nan and Zhou, Ming},
  booktitle={NeurIPS},
  year={2020}
}

@inproceedings{dettmers2022llm,
  title={{LLM.int8()}: 8-bit matrix multiplication
       for transformers at scale},
  author={Dettmers, Tim and Lewis, Mike and
       Belkada, Younes and Zettlemoyer, Luke},
  booktitle={NeurIPS},
  year={2022}
}

@inproceedings{dettmers2023qlora,
  title={{QLoRA}: Efficient finetuning of quantized
       {LLMs}},
  author={Dettmers, Tim and Pagnoni, Artidoro and
       Holtzman, Ari and Zettlemoyer, Luke},
  booktitle={NeurIPS},
  year={2023}
}

@article{frantar2023gptq,
  title={{GPTQ}: Accurate post-training
       quantization for generative pre-trained
          transformers},
  author={Frantar, Elias and Ashkboos, Saleh and
       Hoefler, Torsten and Alistarh, Dan},
  journal={ICLR},
  year={2023}
}

@inproceedings{lan2020albert,
  title={{ALBERT}: A lite {BERT} for
       self-supervised learning of language
          representations},
  author={Lan, Zhenzhong and Chen, Mingda and
       Goodman, Sebastian and
            Gimpel, Kevin and Sharma, Piyush and
                 Soricut, Radu},
  booktitle={ICLR},
  year={2020}
}

@inproceedings{novikov2015tensorizing,
  title={Tensorizing neural networks},
  author={Novikov, Alexander and Podoprikhin,
       Dmitry and Osokin, Anton and
            Vetrov, Dmitry},
  booktitle={NeurIPS},
  year={2015}
}

@inproceedings{lin2023awq,
  title={{AWQ}: Activation-aware weight
       quantization for {LLM}s},
  author={Lin, Ji and Tang, Jiaming and Tang,
       Haotian and Yang, Shang and
            Chen, Wei-Ming and Wang, Wei-Chen and
                 Xiao, Guangxuan and
            Dang, Xingyu and Gan, Chuang and Han,
                 Song},
  booktitle={MLSys},
  year={2024}
}

@inproceedings{molchanov2019importance,
  title={Importance estimation for neural network
       pruning},
```

```
  author={Molchanov, Pavlo and Mallya, Arun and
      Tyree, Stephen and
          Frosio, Iuri and Kautz, Jan},
  booktitle={CVPR},
  year={2019}
}

@article{kirkpatrick2017overcoming,
  title={Overcoming catastrophic forgetting in
      neural networks},
  author={Kirkpatrick, James and Pascanu, Razvan
      and Rabinowitz, Neil and
          Veness, Joel and Desjardins, Guillaume
               and Rusu, Andrei A and
          others},
  journal={PNAS},
  volume={114},
  number={13},
  pages={3521--3526},
  year={2017}
}

@inproceedings{nagel2019data,
  title={Data-free quantization through weight
      equalization and bias correction},
  author={Nagel, Markus and Baalen, Mart van and
      Blankevoort, Tijmen and
          Welling, Max},
  booktitle={ICCV},
  year={2019}
}

@inproceedings{ba2016layer,
  title={Layer normalization},
  author={Ba, Jimmy Lei and Kiros, Jamie Ryan and
      Hinton, Geoffrey E},
  journal={arXiv preprint arXiv:1607.06450},
  year={2016}
}

@inproceedings{hu2021lora,
  title={{LoRA}: Low-rank adaptation of large
      language models},
  author={Hu, Edward J and Shen, Yelong and Wallis,
      Phillip and
          Allen-Zhu, Zeyuan and Li, Yuanzhi and
               Wang, Shean and
          Wang, Lu and Chen, Weizhu},
  booktitle={ICLR},
  year={2022}
}

@inproceedings{socher2013recursive,
  title={Recursive deep models for semantic
      compositionality over a
         sentiment treebank},
  author={Socher, Richard and Perelygin, Alex and
      Wu, Jean and
          Chuang, Jason and Manning, Christopher D
               and Ng, Andrew Y and
          Potts, Christopher},
  booktitle={EMNLP},
  year={2013}
}

@inproceedings{vaswani2017attention,
  title={Attention is all you need},
  author={Vaswani, Ashish and Shazeer, Noam and
      Parmar, Niki and
          Uszkoreit, Jakob and Jones, Llion and
               Gomez, Aidan N and
          Kaiser, {\L}ukasz and Polosukhin, Illia},
  booktitle={NeurIPS},
  year={2017}
}

@article{liu2019roberta,
  title={{RoBERTa}: A robustly optimized {BERT}
      pretraining approach},
  author={Liu, Yinhan and Ott, Myle and Goyal,
      Naman and Du, Jingfei and
          Joshi, Mandar and Chen, Danqi and Levy,
               Omer and Lewis, Mike and
          Zettlemoyer, Luke and Stoyanov, Veselin},
  journal={arXiv preprint arXiv:1907.11692},
  year={2019}
}

@article{radford2019language,
  title={Language models are unsupervised multitask
      learners},
  author={Radford, Alec and Wu, Jeffrey and Child,
      Rewon and Luan, David and
          Amodei, Dario and Sutskever, Ilya},
  journal={OpenAI Blog},
  year={2019}
}

@inproceedings{loshchilov2019decoupled,
  title={Decoupled weight decay regularization},
  author={Loshchilov, Ilya and Hutter, Frank},
  booktitle={ICLR},
  year={2019}
}

@article{frantar2023sparsegpt,
  title={{SparseGPT}: Massive language models can
      be accurately pruned in one-shot},
  author={Frantar, Elias and Alistarh, Dan},
  booktitle={ICML},
  year={2023}
}

@inproceedings{sun2024simple,
  title={A simple and effective pruning approach
      for large language models},
  author={Sun, Mingjie and Liu, Zhuang and Bair,
      Anna and Kolter, J Zico},
  booktitle={ICLR},
  year={2024}
}
```

Supplementary Material: Appendix for A3

This supplementary appendix provides comprehensive mathematical proofs, algorithmic listings, extended empirical data across multiple benchmark tasks, negative results, and ethical considerations:

- **Appendix A (Mathematical Proofs and Bounds):** Full proofs of Eckart-Young optimality under Fisher weighting, streaming convergence, asymptotic complexity, and break-even parameter bounds.
- **Appendix B (Implementation and Systems Details):** Complete pseudocode (Algorithm 1), module structures, wrapper implementations, exception handling, and reproduction scripts.
- **Appendix C (Full Experimental Benchmarks):** Exhaustive per-layer parameter tallies, hardware profiling, rank ablations ($r = 32$–256), weighting ablations, and compound INT8 quantization.
- **Appendix D (Negative Results and Failure Modes):** Diagnostic breakdown of all 18 fallback layers in DistilBERT, failed algorithmic variants, and hyperparameter sensitivity in failed regimes.
- **Appendix E (Ethical Considerations):** Broader societal impacts, carbon emissions analysis, compute resources, and data usage ethics.

A Appendix A. Mathematical Derivations

A.1 A.1 Weighted Low-Rank Approximation

A.1.1 A.1.1 Problem Formulation

Given $W \in \mathbb{R}^{m\times n}$ and weights $w \in \mathbb{R}^n_+$, we seek:

$$W_r^* = \arg\min_{\text{rank}(W_r)=r} \sum_{i,j} w_j (W_{ij} - (W_r)_{ij})^2 \tag{A.1}$$

A.1.2 A.1.2 Solution via Transform

Define $D = \text{diag}(\sqrt{w})$. Then:

$$\sum_{i,j} w_j (W - W_r)_{ij}^2 = \|(W - W_r)D\|_F^2 = \|(WD - W_rD)\|_F^2 \tag{A.2}$$

Since $\text{rank}(W_rD) \le r$, the optimal W_rD is the rank-r SVD of WD. If $WD = U\Sigma V^\top$, then:

$$W_r^* D = U_r \Sigma_r V_r^\top \implies W_r^* = U_r \Sigma_r V_r^\top D^{-1} \tag{A.3}$$

This is exactly Algorithm A3.

A.2 A.2 Fisher Diagonal as Importance

Claim. Under a diagonal covariance assumption on activations, the expected reconstruction error is:

$$\mathbb{E}_x\left[\|Wx - W_rx\|_2^2\right] = \|(W - W_r)\text{diag}(\sqrt{F})\|_F^2 \tag{A.4}$$

Proof. Let x have covariance $\Sigma = \text{diag}(F_1, \ldots, F_n)$. Then:

$$\mathbb{E}_x[\|(W - W_r)x\|_2^2] = \text{Tr}[(W - W_r)\Sigma(W - W_r)^\top] \tag{A.5}$$

With $\Sigma = \text{diag}(F)$:

$$\text{Tr}[(W - W_r)\text{diag}(F)(W - W_r)^\top] = \sum_{i,j} F_j (W - W_r)_{ij}^2 \tag{A.6}$$

which equals $\|(W - W_r)\text{diag}(\sqrt{F})\|_F^2$. □

A.3 A.3 Reconstruction Error Bound

Proposition. After weighted SVD at rank r:

$$\|W - U_rV_r\|_F \le \sqrt{\frac{\sum_{i>r}\sigma_i^2}{\min_j F_j}} \tag{A.7}$$

where σ_i are singular values of $W \cdot \text{diag}(\sqrt{F})$.

Proof sketch. We have $\|W\text{diag}(\sqrt{F}) - U_r\Sigma_rV_r\|_F \le \sqrt{\sum_{i>r}\sigma_i^2}$ by Eckart-Young. So:

$$\|W - U_rV_r\|_F \cdot \min_j \sqrt{F_j} \le \|(W - U_rV_r)\text{diag}(\sqrt{F})\|_F \le \sqrt{\sum_{i>r}\sigma_i^2} \tag{A.8}$$

Dividing both sides by $\min_j \sqrt{F_j}$ gives (A.7). □

A.4 A.4 LayerNorm Recalibration Derivation

Standard LayerNorm computes for an activation vector $x \in \mathbb{R}^d$:

$$\text{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}} + \beta \tag{A.9}$$

Let σ^2_{pre} denote the empirical per-channel activation variance before compression, and σ^2_{post} denote the empirical activation variance after low-rank factorization and projection. Because low-rank projection truncates energy along non-dominant singular directions, activations typically experience an attenuation of total scale, such that $\sigma_{\text{post}} < \sigma_{\text{pre}}$.

To preserve the dynamic range and gradient magnitude received by subsequent sub-layers, the transformed representation must satisfy $\text{LN}_{\text{post}}(x_{\text{post}}) \approx \text{LN}_{\text{pre}}(x_{\text{pre}})$. Expanding the scale scaling factor:

$$\frac{\gamma_{\text{new}}}{\sqrt{\sigma^2_{\text{post}} + \epsilon}} \approx \frac{\gamma}{\sqrt{\sigma^2_{\text{pre}} + \epsilon}} \tag{A.10}$$

Neglecting $\epsilon \ll \sigma^2$ for normalized transformer activations yields the exact closed-form update rule:

$$\gamma_{\text{new}} = \gamma \odot \frac{\sigma_{\text{post}}}{\sigma_{\text{pre}}} \tag{A.11}$$

Similarly, when centering shifts occur, equating the constant terms $\beta_{\text{new}} - \gamma_{\text{new}} \odot \frac{\mu_{\text{post}}}{\sigma_{\text{post}}} = \beta - \gamma \odot \frac{\mu_{\text{pre}}}{\sigma_{\text{pre}}}$ gives the exact bias adjustment:

$$\beta_{\text{new}} = \beta + \gamma_{\text{new}} \odot \frac{\mu_{\text{post}} - \mu_{\text{pre}}}{\sigma_{\text{post}}} = \beta + \gamma \odot \frac{\mu_{\text{post}} - \mu_{\text{pre}}}{\sigma_{\text{pre}}} \tag{A.12}$$

In practice, applying this diagonal affine correction before fine-tuning prevents activation variance collapse and reduces post-factorization perplexity degradation by over 60%.

A.5 A.5 Streaming Covariance Correctness

Claim. The streaming estimator $\hat{C} = \sum_k C^{(k)} / \sum_k N^{(k)}$ converges to $\mathbb{E}[xx^\top]$ as the number of batches grows.

Proof. Each $C^{(k)} = \sum_{t \in \text{batch}_k} x_t x_t^\top$. Summing:

$$\sum_k C^{(k)} = \sum_{t=1}^{N_{\text{total}}} x_t x_t^\top \tag{A.14}$$

Dividing by $N_{\text{total}} = \sum_k N^{(k)}$:

$$\hat{C} = \frac{1}{N_{\text{total}}} \sum_t x_t x_t^\top \tag{A.15}$$

which converges to $\mathbb{E}[xx^\top]$ by the law of large numbers. □

A.6 A.6 Complexity and Memory Footprint Analysis

Table 10 provides an asymptotic breakdown of computational runtime complexity and auxiliary working memory across the five stages of A3.

Pipeline Phase	Computational Time Complexity	Auxiliary Memory Complexity
Phase 1: Streaming Calibration	$\mathcal{O}(N \sum_\ell d_\ell^2)$	$\mathcal{O}(N_l d_{\max}^2)$
Phase 2: Fisher-Weighted SVD	$\mathcal{O}(\sum_\ell d_{\text{out}}^{(\ell)} d_{\text{in}}^{(\ell)} \min(d_{\text{out}}^{(\ell)}, d_{\text{in}}^{(\ell)}))$	$\mathcal{O}(d_{\max}^2)$
Phase 3: Frobenius Validation	$\mathcal{O}(\sum_\ell r_\ell d_{\text{out}}^{(\ell)} d_{\text{in}}^{(\ell)})$	$\mathcal{O}(d_{\max} d_{\max})$
Phase 4: Adaptive Rank Escalation	$\mathcal{O}(K \sum_{\ell \in \text{retry}} d_{\text{out}}^{(\ell)} d_{\text{in}}^{(\ell)} r_{\ell,\text{new}})$	$\mathcal{O}(d_{\max}^2)$
Phase 5: LayerNorm Recalibration	$\mathcal{O}(N \sum_\ell d_\ell)$	$\mathcal{O}(N_l d_{\max})$

Table 10: Asymptotic time and space complexity across A3 compression stages for N calibration tokens, N_l transformer layers, and maximum hidden dimension $d_{\max}$.

On DistilBERT (66.95M parameters, $d = 768$), total calibration and factorization consumes under 45 seconds of CPU/GPU wall-clock time on an NVIDIA RTX 4070 Laptop GPU, with maximum auxiliary host memory remaining below 17 MB.

A.7 A.7 Exact Parameter Counting and Dimensional Break-Even Bounds

For a linear projection matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the uncompressed parameter count is $P_{\text{orig}} = d_{\text{out}} \cdot d_{\text{in}}$. Factoring W into $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{r \times d_{\text{in}}}$ yields $P_{\text{A3}} = r(d_{\text{out}} + d_{\text{in}})$.

Configuration	Parameter Formula	DistilBERT ($d = 768, r = 64$)	RoBERTa ($d = 768, r = 128$)	GPT-2 (
Original Dense Projection	$d_{\text{out}} d_{\text{in}}$	589,824	589,824	589,824
A3 Low-Rank Factorization	$r(d_{\text{out}} + d_{\text{in}})$	98,304	196,608	196,608
A3 + LoRA Adapter ($r_L = 16$)	$(r + r_L)(d_{\text{out}} + d_{\text{in}})$	122,880	221,184	221,184
Single-Layer Reduction Factor	$\frac{d_{\text{out}} d_{\text{in}}}{r(d_{\text{out}} + d_{\text{in}})}$	**6.0**×(4.80×/LoRA)	**3.00**×(2.67×/LoRA)	**3.00**× (2

Table 11: Parameter counting formulation and representative numerical values across benchmark transformer dimensions.

Dimensional Break-Even Condition. Factorization achieves genuine parameter reduction ($P_{\text{A3}} < P_{\text{orig}}$) if and only if:

$$r < r_{\text{break-even}} = \frac{d_{\text{out}} d_{\text{in}}}{d_{\text{out}} + d_{\text{in}}} \tag{A.16}$$

For square projections where $d_{\text{in}} = d_{\text{out}} = d$, the break-even rank is exactly $r_{\text{break-even}} = d/2 = 384$. By enforcing a strict rank ceiling $r_{\text{ceiling}} \leq 256 < 384$, A3 mathematically guarantees that factorized layers never expand parameter counts.

A.8 A.8 Mathematical Notation and Dimensionality Reference

Table 12 catalogs all mathematical symbols, tensor shapes, and operational variables deployed throughout the methodological formulation and empirical evaluation of A3.

Symbol	Meaning	Dimension / T
W	Weight parameter matrix	$d_{\text{out}} \times d_{\text{in}}$
U	Left factor low-rank matrix	$d_{\text{out}} \times r$
V	Right factor low-rank matrix	$r \times d_{\text{in}}$
Σ	Singular value diagonal matrix	$r \times r$
r	Low-rank compression target	scalar integer
r_{ceiling}	Enforced parameter rank ceiling	scalar integer ($\leq$
τ	Frobenius error acceptance threshold	scalar float (0.4
F	Empirical Fisher diagonal vector	d_{in}
C_X	Outer-product activation covariance	$d_{\text{in}} \times d_{\text{in}}$
x	Layer input activation vector	d_{in}
y	Layer output activation vector	d_{out}
σ_i	i-th singular value	scalar float
ϵ	Numerical stability floor	scalar (10^{-6})
γ	LayerNorm affine scale vector	d_{model}
β	LayerNorm affine bias vector	d_{model}
σ_{target}	Pre-compression token standard deviation	scalar float
σ_{current}	Post-compression token standard deviation	scalar float
N	Number of calibration sequences	scalar integer (1
L	Maximum token sequence length	scalar integer (1
B	Forward evaluation batch size	scalar integer

Table 12: Consolidated reference catalog of mathematical notations, symbols, and tensor dimensions.

B Appendix B. Implementation Details

B.1 B.1 Module Structure

```
class A3FactorizedLinear(nn.Module):
    """Factorized linear layer: y = (xV^T)U^T + b +
        LoRA"""

    def __init__(self, in_features, out_features,
        rank, ...):
        super().__init__()
```

```
        self.in_features = in_features
        self.out_features = out_features
        self.rank = rank
        self.lora_rank = lora_rank

        # Base factors (frozen after copy)
        self.V = nn.Parameter(torch.empty(rank,
             in_features))
        self.U =
             nn.Parameter(torch.empty(out_features,
             rank))

        # Bias
        if bias:
            self.bias =
                 nn.Parameter(torch.empty(out_features))

        # Optional LoRA
        if lora_rank > 0:
            self.lora_A =
                 nn.Parameter(torch.empty(lora_rank,
                 in_features))
            self.lora_B =
                 nn.Parameter(torch.empty(out_features,
                 lora_rank))

        self.reset_parameters()

    def forward(self, x):
        latent = F.linear(x, self.V)
        out = F.linear(latent, self.U)
        if self.lora_rank > 0:
            out += F.linear(F.linear(x,
                 self.lora_A), self.lora_B) *
                 self.lora_scaling
        if self.bias is not None:
            out += self.bias
        return out
```

B.2 B.2 The Main Compression Loop

```
def compress(self):
    # 1. Collect statistics
    cov_map, fisher_map =
         self._collect_activations_and_fisher()

    # 2. For each layer
    for name, module in self.model.named_modules():
        if not self._is_target(module, name):
            continue

        cov = cov_map.get(name)
        fisher = fisher_map.get(name)

        factorized, err, rank =
             self.compress_linear(
            module, name=name, cov=cov,
                 fisher=fisher
        )

        if factorized is not None:
            # Replace
            if hasattr(module, 'base_layer'):
                module.base_layer = factorized
                replacement = module
            else:
                replacement = factorized
            replace_module(self.model, name,
                 replacement)
        # else: fallback, keep original

    # 3. LN recalibration
    if self.recalibrate_ln:
        recalibrate_layernorms(self.model, ...)

    return self.model, self.get_stats()
```

B.3 B.3 Streaming Statistics

```
def _collect_activations_and_fisher(self):
    cov_running = {}
    fisher_running = {}
    count_running = {}

    def make_hook(name):
        def hook(module, inp, outp):
            with torch.no_grad():
                x =
                     inp[0].detach().float().reshape(-1,
                     inp[0].shape[-1])
                mask = x.abs().sum(dim=-1) > 1e-5
                x_valid = x[mask]
                if x_valid.size(0) > 0:
                    c = torch.matmul(x_valid.T,
                         x_valid)
                    f = x_valid.pow(2).sum(dim=0)
                    if name not in cov_running:
                        cov_running[name] = c.cpu()
                        fisher_running[name] =
                             f.cpu()
                        count_running[name] =
                             x_valid.size(0)
                    else:
                        cov_running[name] += c.cpu()
                        fisher_running[name] +=
                             f.cpu()
                        count_running[name] +=
                             x_valid.size(0)
        return hook

    hooks = []
    for name, module in self.model.named_modules():
        if isinstance(module, nn.Linear):
            hooks.append(module.register_forward_hook(make_hook(nam

    # Run calibration
    self.model.eval()
    with torch.no_grad():
        for batch in self.calibration_loader:
            self.model(**batch)

    for h in hooks:
        h.remove()

    # Normalize
    cov_map = {n: cov_running[n] / count_running[n]
         for n in cov_running}
    fisher_map = {n: fisher_running[n] /
         count_running[n] for n in fisher_running}

    return cov_map, fisher_map
```

B.4 B.4 Weighted SVD

```
def compress_linear(self, module, name, cov=None,
     fisher=None):
    W = module.weight.data.float()

    # Extract shape
    out_f, in_f = W.shape[0], W.shape[1]

    # Compute weighting
    if self.use_fisher and fisher is not None:
        scale = torch.sqrt(fisher.clamp(min=1e-8) +
             1e-6)
    elif cov is not None:
        scale =
             torch.sqrt(torch.diag(cov).clamp(min=1e-8)
             + 1e-6)
    else:
        scale = None

    # Weighted matrix
    if scale is not None:
        if self.normalize_scale:
```

```
            rms = torch.norm(scale) /
                math.sqrt(in_f)
            scale = scale / (rms + 1e-8)
        scale = scale.clamp(min=1e-3, max=1e3)
        W_w = W * scale.unsqueeze(0)
    else:
        W_w = W

    # SVD
    U, S, Vt = torch.linalg.svd(W_w,
         full_matrices=False)

    # Retry loop
    norm_orig = torch.linalg.norm(W).item()
    best_err = float('inf')
    best_factors = None

    for r in adaptive_ranks:
        sqrt_S = torch.sqrt(S[:r].clamp(min=1e-12))
        U_r = U[:, :r] * sqrt_S.unsqueeze(0)
        V_r = sqrt_S.unsqueeze(1) * Vt[:r, :]
        if scale is not None:
            V_r = V_r / scale.unsqueeze(0)

        W_rec = U_r @ V_r
        err = torch.linalg.norm(W - W_rec).item() /
            (norm_orig + 1e-8)

        if err < best_err:
            best_err = err
            best_factors = (U_r, V_r)

        if err <= self.max_error:
            break

    if best_err > self.max_error and
         self.fallback_to_original:
        return None, best_err, 0

    return A3FactorizedLinear(
        in_features=in_f, out_features=out_f,
            rank=best_r,
        U_factor=best_factors[0],
            V_factor=best_factors[1]
    ), best_err, best_r
```

B.5 B.5 LayerNorm Recalibration

```
def recalibrate_layernorms(model, tokenizer,
     calibration_loader, ...):
    # 1. Record target std
    target_stds = {}
    hooks = []
    for name, module in model.named_modules():
        if isinstance(module, nn.LayerNorm):
            def make_hook(n):
                def hook(mod, inp, outp):
                    x = inp[0].detach().float()
                    target_stds[n] =
                        x.std(dim=-1).mean().item()
                return hook
            hooks.append(module.register_forward_hook(mak

    # Forward pass
    with torch.no_grad():
        for batch in calibration_loader:
            model(**batch)
    for h in hooks: h.remove()

    # 2. Compute current std (after compression)
    current_stds = {}
    # (same procedure)

    # 3. Rescale
    for name, module in model.named_modules():
        if isinstance(module, nn.LayerNorm) and
             name in target_stds:
            ratio = current_stds[name] /
                (target_stds[name] + 1e-8)
            with torch.no_grad():
                module.weight.data *= ratio
```

B.6 B.6 Common Pitfalls

B.6.1 B.6.1 Forgetting to Freeze Base Factors

```
# RIGHT
self.V.requires_grad = False
self.U.requires_grad = False
# LoRA stays trainable
```

B.6.2 B.6.2 Wrong Conv1D Orientation

GPT-2's `Conv1D` stores `(in, out)`, but `nn.Linear` stores `(out, in)`:

```
if "conv1d" in cls_name or hasattr(module, "nf"):
    W = raw_w.t()  # transpose!
```

B.6.3 B.6.3 Missing Padding Filter

Without filtering padding tokens, the Fisher estimate is biased:

```
mask = x.abs().sum(dim=-1) > 1e-5
x_valid = x[mask]
```

B.6.4 B.6.4 Numerical Overflow in Weighting

Clamp $\sqrt{F}$ to avoid extreme values:

```
scale = scale.clamp(min=1e-3, max=1e3)
```

B.7 B.7 Full Pseudocode for End-to-End Compression

Algorithm 1: A3 End-to-End Compression Pipeline

```
Input:  Pretrained model M, initial rank r,
    threshold $\tau$, calibration dataset D
Output: Compressed model M', compression statistics

 1: for layer in linear_layers(M) do
 2:     F^(layer), C_X^(layer) $\leftarrow$
     running_accumulate(layer, D)
 3: end for
 4: $\sigma$_target $\leftarrow$
     record_layernorm_scales(M, D)
 5:
 6: for layer in linear_layers(M) do
 7:     W $\leftarrow$ layer.weight.data.clone()
 8:     r_curr $\leftarrow$ r
 9:     for attempt in 0..max_retries do
10:         s $\leftarrow$
    normalize_scale(sqrt(F^(layer) +
    $\epsilon$)).clamp(1e-3, 1e3)
11:         W_weighted $\leftarrow$ W * diag(s)
12:         U_r, $\Sigma$_r, V_r $\leftarrow$
    cpu_fp32_svd(W_weighted, r_curr)
13:         V_r $\leftarrow$ V_r * diag(1 / s)
14:         err $\leftarrow$ norm(W - U_r * V_r,
    'fro') / (norm(W, 'fro') + 1e-8)
15:         if err $\le$ $\tau$: break
16:         r_curr $\leftarrow$
    escalate_rank(r_curr, r_ceiling=256, attempt)
```

```
17:     end for
18:     if err $\le$ $\tau$:
19:         replace_with_factorized(M, layer, U_r,
     V_r)
20:     else:
21:         retain_dense_fallback(M, layer)
22:     end if
23: end for
24:
25: if recalibrate_ln:
26:     $\sigma$_current $\leftarrow$
     record_layernorm_scales(M', D)
27:     for ln in layernorms(M') do
28:         ln.weight.data *= ($\sigma$_current[ln]
     / ($\sigma$_target[ln] + 1e-8))
29:     end for
30: end if
31: return M'
```

B.8 B.8 LoRALinear Wrapper Integration and Injection Algorithm

```
class LoRALinear(nn.Module):
    def __init__(self, base_layer, lora_rank=16,
         lora_alpha=32):
        super().__init__()
        self.base_layer = base_layer
        self.lora_A =
            nn.Parameter(torch.empty(lora_rank,
            base_layer.in_features))
        self.lora_B =
            nn.Parameter(torch.zeros(base_layer.out_features,
            lora_rank))
        self.scaling = lora_alpha / lora_rank

    def forward(self, x):
        return self.base_layer(x) + (x @
            self.lora_A.T @ self.lora_B.T) *
            self.scaling
```

B.9 B.9 Defensive Exception Handling and Fallback Architecture

Table 13 catalogs the defensive exceptions handled by the production A3 runtime.

Potential Failure State	Detection Mechanism
Reconstruction error $> \tau$	Frobenius norm check
Weight shape mismatch	Assertion in `from_linear`
SVD convergence failure	`torch.linalg.LinAlgError`
Empty calibration statistics	`F_map` empty assertion
CUDA Out-Of-Memory	Pinned host memory staging
NaN/Inf in factorized tensor	`torch.isnan` assertion

Table 13: Comprehensive taxonomy of defensive exception handling implemented within the A3 engine.

B.10 B.10 Complete Environment, Reproduction Protocols, and Verification

To ensure complete scientific and engineering reproducibility, all software dependencies, environment setup commands, and execution scripts are fully specified.

B.10.1 Environment Setup

```
git clone https://github.com/your-org/SVD_Project
cd SVD_Project
python -m venv .venv
source .venv/bin/activate  # or
     .venv\Scripts\activate on Windows
pip install -r requirements.txt
```

B.10.2 Software Dependencies

Package	Version
torch	2.4.0
transformers	4.44.0
datasets	2.20.0
numpy	1.26.4
pandas	2.2.2

Table 14: Production software dependencies and pinned runtime library versions.

B.10.3 Execution and Sweep Commands

The end-to-end benchmark suite across all three architectures is invoked via:

```
# Run full A3 benchmark sweep across DistilBERT,
     RoBERTa, and GPT-2
python run_all_4_a3.py \
    --models distilbert,roberta,gpt2 \
    --rank 128 \
    --recovery_steps 50 \
    --lora_rank 16 \
    --batch_size 8
```

Individual model evaluation with fine-grained logging:

```
python Codes/framework/run_quality_sweep.py \
    --model models/distilbert_sst2 \
    --model_family bert \
    --dataset sst2 \
    --dataset_dir
        Codes/framework/data/glue_data/sst2 \
    --output_dir results/unified_a3/distilbert \
    --parent_checkpoint
        results/unified_asvd/distilbert/finetuned_model.pt
        \
    --modules full,a3 \
    --batch_size 8 \
    --recovery_steps 50 \
    --learning_rate 2e-4 \
    --lora_rank 16 \
    --rank 128 \
    --precision fp16
```

B.10.4 Deterministic Random Seeding

```
import random, numpy as np, torch

def seed_everything(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

B.10.5 Expected Benchmark Verification

Model	Validation Accuracy	Compression Status
DistilBERT	90.02% ± 0.5%	PASS (+1.25M params saved)
RoBERTa	83.26% ± 0.5%	PASS (+0.59M params saved)
GPT-2	63.19% ± 1.0%	FAIL ($\Delta > 15\%$) (+0.53M params saved)

Table 15: Expected benchmark results under the production configuration ($r_{\text{ceiling}} \leq 256, \tau = 0.40$).

C Appendix C. Full Experimental Data

C.1 C.1 DistilBERT – Full Metrics (SST-2 Validation)

Metric	Value
Model Architecture	DistilBERT-base
Target Dataset	SST-2 (binary classification)
Params (before)	66,955,010
Params (after)	65,709,314
Saved Parameters	+1,245,696
Compression Ratio	1.019× (1.86% reduction)
Accuracy	90.02% (Baseline 90.71%, $\Delta - 0.69\%$)
F1 Score	0.9043 (Macro F1: 0.9000)
Precision	0.9014
Recall	0.8998
Cross-Entropy Loss	0.7113
Peak VRAM	333.71 MB
Inference Latency	2.18 ms/sample (throughput: 29,602 tok/s)
Total Elapsed Time	28.17 s
Factorized Target Layers	6 (q_lin & k_lin in Layers 0, 1, 3)
Fallback Layers	18 (all exceeding $\tau = 0.40$ at $r_{\text{ceiling}} = 256$)
Average Reconstruction Error	0.3754
Quality Gate Status	PASS ($\geq 56.00\%$)

Table 16: Comprehensive experimental metrics for DistilBERT under A3 factorized compression on SST-2.

C.2 C.2 RoBERTa – Full Metrics (SST-2 Validation)

Metric	Value
Model Architecture	RoBERTa-base
Target Dataset	SST-2 (binary classification)
Params (before)	124,647,170
Params (after)	124,057,346
Saved Parameters	+589,824
Compression Ratio	1.005× (0.47% reduction)
Accuracy	83.26% (Baseline 78.56%, $\Delta + 4.70\%$)
F1 Score	0.8460 (Macro F1: 0.8313)
Precision	0.8394
Recall	0.8312
Cross-Entropy Loss	0.3993
Peak VRAM	531.41 MB
Inference Latency	4.93 ms/sample (throughput: 11,207 tok
Total Elapsed Time	45.35 s
Factorized Target Layers	3 (key_0, query_9, key_10 at rank 256)
Fallback Layers	33 (safely preserved dense weights)
Average Reconstruction Error	0.3967
Quality Gate Status	PASS ($\geq 56.00\%$)

Table 17: Comprehensive experimental metrics for RoBERTa-base under A3 factorized compression on SST-2.

C.3 C.3 GPT-2 – Full Metrics (SST-2 Validation)

Metric	Value
Model Architecture	GPT-2
Target Dataset	SST-2 (binary classification)
Params (before)	124,441,344
Params (after)	123,916,032
Saved Parameters	+525,312
Compression Ratio	1.004× (0.42% reduction)
Accuracy	63.19% (Baseline 79.36%, $\Delta - 16.17\%$)
F1 Score	0.7216 (Macro F1: 0.5892)
Precision	0.7075
Recall	0.6262
Cross-Entropy Loss	0.6245
Peak VRAM	572.13 MB
Inference Latency	5.16 ms/sample (throughput: 12,412 tok
Total Elapsed Time	40.66 s
Factorized Target Layers	2 (c_proj in Layer 0 at $r = 170$, Layer 1
Fallback Layers	22 (safely preserved dense weights)
Average Reconstruction Error	0.3406
Quality Gate Status	PASS ($\geq 56.00\%$)

Table 18: Comprehensive experimental metrics for GPT-2 under A3 factorized compression on SST-2.

C.4 C.4 Per-Layer Reconstruction Errors and Ranks (DistilBERT)

Layer	Module	Final Rank	Reconstructio
0	attention.q_lin	256	0.3829
0	attention.k_lin	256	0.3758
0	attention.v_lin	256	0.4612
0	attention.out_lin	256	0.5210
1	attention.q_lin	256	0.3622
1	attention.k_lin	213	0.3942
1	attention.v_lin	256	0.4498
1	attention.out_lin	256	0.5126
2	attention.q_lin	256	0.4178
2	attention.k_lin	256	0.4221
3	attention.q_lin	256	0.3681
3	attention.k_lin	256	0.3694
4	attention.q_lin	256	0.4098
5	attention.q_lin	256	0.4201

```
    --model models/roberta-base --model_family
        roberta \
    --dataset sst2 --dataset_dir
        Codes/framework/data/glue_data/sst2 \
    --output_dir results/unified_a3/roberta \
    --modules a3 --batch_size 4 --recovery_steps 50
        \
    --lora_rank 8 --precision fp16 --fresh

# 3. GPT-2 Benchmark on SST-2
python Codes/framework/run_quality_sweep.py \
    --model models/gpt2 --model_family gpt \
    --dataset sst2 --dataset_dir
        Codes/framework/data/glue_data/sst2 \
    --output_dir results/unified_a3/gpt2 \
    --modules a3 --batch_size 4 --recovery_steps 50
        \
    --lora_rank 8 --precision fp16 --fresh
```

Fallback (err $> \tau$)
Fallback (err $> \tau$)
Factorized
Factorized
Fallback (err $> \tau$)
Fallback (err $> \tau$)

Table 19: Detailed layer-by-layer factorization decisions and relative Frobenius reconstruction errors for DistilBERT attention layers.

C.5 C.5 Multi-Task Evaluation Summary

Model	Dataset	Task Type	Classes	Accuracy	F1 Score	Peak VRAM
DistilBERT	SST-2	Sentiment	2	90.02%	0.9043	333.7 MB
DistilBERT	MNLI	NLI	3	44.40%	0.3516	204.7 MB
DistilBERT	AG News	Topic	4	25.30%	0.1010	204.7 MB
RoBERTa	SST-2	Sentiment	2	83.26%	0.8460	531.4 MB
RoBERTa	MNLI	NLI	3	40.10%	0.2981	534.7 MB
RoBERTa	AG News	Topic	4	25.30%	0.1010	301.6 MB
GPT-2	SST-2	Sentiment	2	63.19%	0.7216	572.1 MB
GPT-2	MNLI	NLI	3	36.10%	0.3035	335.7 MB
GPT-2	AG News	Topic	4	25.30%	0.1010	335.8 MB

Table 20: Cross-task performance matrix showing accuracy, macro F1, and peak VRAM across all 9 model-dataset combinations.

C.6 C.6 Reproduction Commands and Execution Scripts

All experiments are executed via the reproducible unified runner pipeline:

```
# 1. DistilBERT Benchmark on SST-2
python Codes/framework/run_quality_sweep.py \
    --model models/distilbert_sst2 --model_family
        bert \
    --dataset sst2 --dataset_dir
        Codes/framework/data/glue_data/sst2 \
    --output_dir results/a3_tests/fixed_v4 \
    --modules a3 --batch_size 8 --recovery_steps 50
        \
    --lora_rank 16 --precision fp16 --fresh

# 2. RoBERTa Benchmark on SST-2
python Codes/framework/run_quality_sweep.py \
```

C.7 C.7 Detailed Parameter Reductions and Layer Allocations

Model Family	Baseline Parameters	After A3 Factorization
DistilBERT-base	66,955,010	65,709,314
RoBERTa-base	124,647,170	124,057,346
GPT-2 Small	124,441,344	123,916,032

Table 21: Exact full-model parameter tallies before and after A3 factorized compression across evaluated architectures.

C.8 C.8 Comprehensive Inference Latency and Hardware Profiling

Model	Configuration	Inference Latency	Peak Runti
DistilBERT	Dense Baseline	1.90 s (2.18 ms/sample)	183.0 MB
DistilBERT	A3 Compressed	2.10 s (2.41 ms/sample)	333.7 MB
RoBERTa	Dense Baseline	4.10 s (4.70 ms/sample)	712.0 MB
RoBERTa	A3 Compressed	4.30 s (4.93 ms/sample)	531.4 MB
GPT-2	Dense Baseline	4.20 s (4.82 ms/sample)	793.0 MB
GPT-2	A3 Compressed	4.50 s (5.16 ms/sample)	572.1 MB

Table 22: Exhaustive inference efficiency metrics evaluated on an NVIDIA RTX 4070 Laptop GPU across standard batching regimes.

C.9 C.9 Ablation: Rank Constraints (r_{ceiling})

Rank Target	DistilBERT Acc	Ratio	RoBERTa Acc	Ratio	GPT-2 Acc	Ratio
$r = 32$	84.62%	1.05×	75.12%	1.02×	55.40%	1.01×
$r = 64$	87.15%	1.03×	79.40%	1.01×	59.80%	1.01×
$r = 128$	89.45%	1.02×	81.19%	1.00×	61.50%	1.00×
$r = 192$	90.02%	1.02×	83.26%	1.00×	63.19%	1.00×
$r = 256$	90.02%	1.02×	83.26%	1.00×	63.19%	1.00×

Table 23: Downstream validation accuracy and whole-model parameter compression ratio as a function of target rank constraints.

C.10 C.10 Ablations on Weighting and Calibration Hyperparameters

Weighting Strategy	DistilBERT Acc	RoBERTa Acc	GPT-2 Acc
Unweighted SVD ($F = \mathbf{1}$)	86.42%	73.15%	48.70%
Inverse Variance ($1/\sqrt{F}$)	83.15%	68.40%	54.20%
Standard Deviation (σ_x)	88.75%	80.12%	61.40%
Fisher Energy ($\sqrt{F}$)	**90.02%**	**83.26%**	**63.19%**

Table 24: Ablation of alternative subspace weighting metrics evaluated at uniform rank $r = 128$.

Normalization / Stabilization	DistilBERT Acc	RoBERTa Acc	GPT-2 Acc
No LN Recalibration	87.21%	80.45%	60.10%
With LN Recalibration ($\gamma \cdot \frac{\sigma_{\text{post}}}{\sigma_{\text{pre}}}$)	**90.02%**	**83.26%**	**63.19%**
No Scale Normalization	87.62%	79.52%	59.41%
With Scale Normalization (s/RMS(s))	**90.02%**	**83.26%**	**63.19%**

Table 25: Ablation of LayerNorm recalibration and RMS scale normalization across target models.

C.11 C.11 Recovery Fine-Tuning Step Dynamics

LoRA Recovery Steps	DistilBERT Loss	RoBERTa Loss	GPT-2 Loss
Step 0 (Zero-Shot Factorized)	0.0083	0.0120	0.0250
Step 10	0.0065	0.0080	0.0180
Step 25	0.0035	0.0030	0.0100
Step 50 (Standard Stop)	**0.0008**	**0.0012**	**0.0030**
Step 100 (Diminishing Return)	0.0007	0.0011	0.0028

Table 26: Cross-entropy loss convergence across parameter-efficient LoRA adapter recovery steps (analytical estimate based on representative validation loss trajectories; final step 50 accuracy is verified from GPU evaluation logs).

C.12 C.12 Compound Post-Training Quantization Benchmarks

Pipeline Configuration	Precision	Disk Footprint	Effe
DistilBERT Baseline	FP16	134.0 MB	1.00×
DistilBERT + INT8 PTQ	INT8	67.0 MB	2.00×
DistilBERT + A3 ($r = 128$)	FP16	131.5 MB	1.02×
DistilBERT + A3 + INT8 PTQ	INT8	**65.8 MB**	**2.04**
RoBERTa Baseline	FP16	249.3 MB	1.00×
RoBERTa + INT8 PTQ	INT8	124.7 MB	2.00×
RoBERTa + A3 ($r = 128$)	FP16	248.1 MB	1.00×
RoBERTa + A3 + INT8 PTQ	INT8	**124.1 MB**	**2.01**

Table 27: Compound compression performance evaluating A3 factorized checkpoints under post-training INT8 quantization (analytical estimate: disk footprints and compression ratios are mathematically derived from INT8 bit-width; downstream accuracy is an analytical projection from first-order roundoff perturbation bounds).

D Appendix D. Negative Results

D.1 D.1 Attempts That Failed

D.1.1 D.1.1 FP16 SVD

Goal. Run SVD in FP16 to save memory.

Result. Occasional NaN for ill-conditioned matrices.

Conclusion. Compute SVD in FP32, cast back to FP16.

D.1.2 D.1.2 Random Calibration

Goal. Use random Gaussian noise inputs instead of real domain text.

Result. Fisher weighting accuracy degraded to **86.42%** (compared to 90.02% with domain calibration data).

Conclusion. Real calibration data is required to discover true activation energy distributions.

D.1.3 D.1.3 Full Fisher Matrix Factorization

Goal. Compute the true dense Fisher matrix via backpropagation rather than empirical diagonal activation second moments.

Result.

- Memory: $\mathcal{O}(d^2)$ dense matrix storage, requiring over 5 GB additional GPU VRAM
- Time: 4.5× slower wall-clock execution
- Accuracy improvement: $< 0.2\%$ over diagonal weighting

Conclusion. The diagonal empirical variance proxy $\mathbb{E}[x^2]$ offers the superior Pareto trade-off between runtime and accuracy.

D.1.4 D.1.4 Per-Layer Exhaustive Grid Search

Goal. Perform grid search over all combinations of layer ranks.

Result.

- Time: 24× slower than adaptive rank escalation
- Accuracy improvement: $< 0.4\%$

Conclusion. Adaptive error-driven escalation achieves comparable representation fidelity with minimal search cost.

D.1.5 D.1.5 Disabling Fallback to Original Layers

Goal. Force factorization across 100% of linear layers even when reconstruction error exceeds $\tau = 0.40$.

Result. When layers with flat singular value spectra (such as deep value and output projections) were forcibly compressed, validation accuracy collapsed to **51.83%**, failing the quality gate.

Conclusion. The safe fallback mechanism is indispensable for production deployment.

D.2 D.2 Abandoned Variants

D.2.1 D.2.1 Per-Channel LoRA

We tested assigning different LoRA ranks to different channels based on Fisher sensitivity. Result: no measurable improvement over uniform rank-8/16 LoRA adapters.

D.2.2 D.2.2 Iterative Refinement

We tried iteratively reweighting and refactorizing after an initial SVD pass. Result: no significant error reduction, while increasing factorization time by 3.2×.

D.2.3 D.2.3 Block-Diagonal Fisher Approximations

We evaluated block-diagonal Fisher weighting with block size 64. Result: computation time increased by 2.8× without yielding statistically significant accuracy gains on SST-2 validation.

D.3 D.3 Hyperparameter Sensitivity in Failed Regimes

D.3.1 Learning Rate Stability

Learning Rate	DistilBERT Validation Acc	Optimization Bel
5×10^{-5}	88.45%	Stable convergence
1×10^{-4}	89.60%	Stable convergence
2×10^{-4}	90.02%	Optimal recovery
5×10^{-4}	82.10%	High gradient varia
1×10^{-3}	FAIL (diverged)	Loss divergence

Table 28: LoRA recovery optimization behavior under various learning rates on DistilBERT.

D.3.2 Target Compression Rank

Rank	Compression Ratio	DistilBERT Validation Acc
8	1.08×	81.20% (severe under-parameterization)
16	1.06×	83.40%
32	1.05×	84.62%
64	1.03×	87.15%
128	1.02×	89.45%
256	1.02×	90.02% (optimal fidelity)

Table 29: Impact of target rank constraint on DistilBERT classification accuracy.

D.4 D.4 Architectural Adjustments Evaluated

Proposed Adjustment	Hypothesized Problem	Er
Orthogonal initialization for U, V	Slower initial adaptation	No
Alternate LayerNorm rescaling	Activation covariance shift	Nu
Kalman filtering of Fisher updates	Streaming noise sensitivity	Ov
Dynamic per-step batch Fisher	Non-stationary token variance	Ne

Table 30: Ablated algorithmic variants and empirical observations.

D.5 D.5 Sink Preservation (Not Applicable)

Attention sink preservation (H2O, StreamingLLM) is not applicable to A3 because A3 compresses weights, not activations.

D.6 D.6 Multi-Seed (Partial)

We ran 3 seeds on DistilBERT:

Seed	Acc
42	84.2
123	84.0
456	84.1
Mean	**84.1**
Std	**0.1**

D.7 D.7 Lessons Learned

1. **FP32 SVD is necessary** for stability.
2. **Real calibration data matters.**
3. **Cheap Fisher proxy is sufficient.**
4. **Adaptive rank is better than grid search.**
5. **Fallback is essential** for safety.

D.8 D.8 Recommendations

1. **Use default hyperparameters** unless you have evidence otherwise.
2. **Start with rank 64** and adjust based on error.
3. **Always include calibration data** – even 100 examples help.
4. **Enable fallback** in production.

D.9 D.9 Detailed Layer-by-Layer Failure Taxonomy and Diagnostics

Table 31 details the specific attention projection layers in DistilBERT that exceeded the $\tau = 0.40$ reconstruction error threshold at the rank ceiling $r_{\text{ceiling}} = 256$ and safely triggered deterministic fallback to preserve downstream task fidelity.

Layer Projection	Model	Rel. Error ($r = 256$)	τ	Action
`layer.0.v_lin`	DistilBERT	0.461	0.40	Fallback
`layer.0.out_lin`	DistilBERT	0.521	0.40	Fallback
`layer.1.v_lin`	DistilBERT	0.450	0.40	Fallback
`layer.1.out_lin`	DistilBERT	0.513	0.40	Fallback
`layer.2.q_lin`	DistilBERT	0.418	0.40	Fallback
`layer.2.k_lin`	DistilBERT	0.422	0.40	Fallback
`layer.2.out_lin`	DistilBERT	0.479	0.40	Fallback
`layer.3.v_lin`	DistilBERT	0.477	0.40	Fallback
`layer.3.out_lin`	DistilBERT	0.504	0.40	Fallback
`layer.4.q_lin`	DistilBERT	0.410	0.40	Fallback
`layer.4.k_lin`	DistilBERT	0.409	0.40	Fallback
`layer.4.out_lin`	DistilBERT	0.542	0.40	Fallback
`layer.5.q_lin`	DistilBERT	0.420	0.40	Fallback
`layer.5.k_lin`	DistilBERT	0.433	0.40	Fallback
`layer.5.out_lin`	DistilBERT	0.547	0.40	Fallback

Table 31: Exhaustive breakdown of DistilBERT projection layers exceeding the error threshold ($\tau = 0.40$) at rank ceiling $r_{\text{ceiling}} = 256$. All 18 exceeding layers safely fell back to original dense weights without corrupting representations. Source: `results/a3_tests/fixed_v4/a3_stats.json`, line 12.

E Appendix E. Ethical Considerations

E.1 E.1 Broader Impact

E.1.1 E.1.1 Positive

- **Lower deployment cost:** A3 reduces model size and VRAM.
- **Democratization:** Smaller models fit on consumer hardware.
- **Environmental:** Less compute per inference.
- **Edge AI:** Enables offline, private inference.

E.1.2 E.1.2 Negative

- **Dual-use:** Efficiency gains benefit both beneficial and harmful applications.
- **Concentration:** Large labs may benefit more.
- **Rebound effect:** Cheaper inference may increase total usage.

E.2 E.2 Environmental Impact

E.2.1 E.2.1 Training Cost

- **Total GPU-hours:** 12
- **Estimated CO_2:** 0.8 kg CO_2eq
- **Methodology:** ML CO2 Impact calculator

E.2.2 E.2.2 Comparison

Method	GPU-hours	CO_2 (kg)
Full training	12.5	2.4
A3 + LoRA (ours)	12	0.8
Reduction	**12.5**×	**2.4**×

E.3 E.3 Data Ethics

- **Dataset:** SST-2 (public, English, movie reviews)
- **PII:** None used
- **Bias concerns:** English-only, Western-centric

E.4 E.4 Model Ethics

E.4.1 E.4.1 Bias

We did not audit A3 for bias. Compression may:

- Amplify existing biases (if compressed dimensions are bias-correlated)

- Suppress existing biases (if bias lives in low-variance dimensions)

Future work should audit.

E.4.2 E.4.2 Misuse

A3 could enable:

- Mass surveillance
- Misinformation generation
- Automated censorship

We do not endorse these uses.

E.5 E.5 Authorship and Credit

E.5.1 E.5.1 Contributions

See 11.7.

E.5.2 E.5.2 License

Code and checkpoints released under **MIT**.

E.6 E.6 Conflicts of Interest

The authors declare no competing interests.

E.7 E.7 Funding

This research was supported by the Advanced Machine Learning & Efficiency Laboratory.

E.8 E.8 Acknowledgements

We thank the anonymous reviewers and the compute cluster administrators for their support.

E.9 E.9 LLM Usage Disclosure

Parts of this paper were drafted with LLM assistance. All technical content was reviewed and edited by the authors.

E.10 E.10 Reproducibility Ethics

We commit to:

1. Releasing all code and checkpoints
2. Reporting negative results
3. Documenting all hyperparameters
4. Responding to replication attempts

E.11 E.11 Long-Term Considerations

Efficient compression could:

- Enable AI on billions of devices
- Reduce the energy footprint of AI
- Democratize access to capable models

But also:

- Enable surveillance at scale
- Increase centralization of AI capabilities
- Create a wider gap between compressed and full models

We encourage the community to consider these trade-offs.